

Natural Language Processing

UNIT 1 — Introduction to Language

Evolution · NLP Basics · Preprocessing · Tokenization · Stemming · POS Tagging · NER

Syllabus

Sr. No.	Topic	Actual Teaching Hours	Contact Hours Equivalence
1	Sequence Models Recurrent Neural Networks and Language Models, N-Gram, Vanishing Gradients, Variants of RNN LSTM, GRU.	10	10
2	Introduction and Basics Data Preprocessing techniques, one-hot encoding methods, Tokenization, embedding, stemming, parts of speech tagging, normalization, names entity recognition, Probabilistic language modeling, Nave Bayes and its applications. Markov models, N-grams, Estimating the probability of a word, and smoothing. Generative models of language. Their application to building an automatically-trained email spam filter, and automatically determining the language, LSI	10	10
3	Text Classification and Generation Introduction, applications , evaluation metrics	7	7
4	Words Embeddings Introduction and Word Vectors word2Vec, Gensim word vectors example, Word Vectors 2 and Word Senses GloVe, Word Window Classification, Neural Networks, Matrix Calculus and Backpropagation, Linguistic Structure: Dependency Parsing, Negative Sampling	8	8
5	Machine Translation Seq2Seq and Attention, BLUE, introduction to Transformer architectures, ConvNets for NLP, Information from parts of words Subword Models, Contextual Word Representations: BERT	10	10
Total		45	45

Course Map

From basic language processing to modern language AI systems

Unit I Language basics and NLP pipeline

clean text, tokenize, POS, NER

Unit II Feature extraction

BoW, TF-IDF, N-grams, Naive Bayes

Unit III Word embeddings

Word2Vec, GloVe, semantic similarity

Unit IV RNNs and sequence models

language models, LSTM, GRU

Unit V Machine translation and modern NLP

Seq2Seq, BERT, GPT, conversational AI

Course Outcomes

What you should be able to do by the end

CO1

Explain evolution, need, applications, and phases of NLP.

CO2

Apply feature extraction methods such as BoW, TF-IDF, N-grams, and probabilistic models.

CO3

Use and compare word embeddings and neural representations.

CO4

Analyze RNN/LSTM/GRU and advanced NLP models for language tasks.

CO5

Understand machine translation and discuss current NLP research systems.

Evaluation Plan

Assessment is linked with course outcomes

CA1 Quiz

CO1

Unit 1: Introduction to Language

10 marks scaled to 4

Tentative: 03/08/2026

CA2 Unit Test

CO2 and CO3

Unit 2: Feature Extraction

Unit 3: Word Embeddings

20 marks scaled to 15

Tentative: 07/09/2026 to
12/09/2026

CA3 Case Studies

CO4 and CO5

Advanced NLP models

Machine translation

12 marks

Tentative: 22/10/2026

Tip: build a small NLP example portfolio while learning. It makes CA3 easier.

Evaluation Scheme

Sr. No.	Component	CO	Marks	Weightage	Tentative Date
CA1	Case Study Based Think Pair Share	CO-1	7	15.3% (4.6/30)	1st week of August 2025
CA2	Midterm Test (CO2:10, CO3:10)	CO-2 CO-3	20	44.3% (13.3/30)	8th to 13th Sept 2025
CA3	Research Assignment	CO-4 CO-5	18	40% (12/30)	27th October 2025

Mode of Communication

➤ Moodle

Reference Books

O'REILLY®

Practical Natural Language Processing

A Comprehensive Guide to Building
Real-World NLP Systems



Real-World

Natural Language Processing

Masato Hagiwara



Online NLP Tutorials

S/N	Tutorial Name	Provider
1.	Natural Language Processing Tutorial	TutorialsPoint
2.	Natural Language Processing	Kaggle
3.	NLP Tutorial	JavaTpoint
4.	Natural Language Processing (NLP) Tutorial	Great Learning
5.	What is Natural Language Processing?	MLTUT
6.	Basics of Natural Language Processing(NLP)	Analytics Vidhya
7.	Natural Language Processing Tutorial	Guru99
8.	NLP Tutorial AI with Python	DataFlair
9.	What Is Natural Language Processing?	Machine Learning Mastery
10.	Natural Language Processing With Python's NLTK Package	Real Python

Best NLP Certificate Courses

[1. Natural Language Processing Specialization- Coursera](#)

[2. Become a Natural Language Processing Expert- Udacity](#)

[3. Natural Language Processing- Coursera](#)

[4. Natural Language Processing in TensorFlow- Coursera](#)

[5. Introduction to Natural Language Processing in Python– DataCamp](#)

[6. Natural Language Processing with Deep Learning in Python -Udemy](#)

[7. Learn Natural Language Processing- Codecademy](#)

[8. Data Science: Natural Language Processing \(NLP\) in Python -Udemy](#)

[9. NLP -Natural Language Processing with Python- Udemy](#)

[10. Natural Language Processing with Python Certification Course- Edureka](#)

Rank	Platform	Free Tier	Best For
1	Hugging Face Spaces	Yes	NLP, LLM demos, Gradio, Streamlit
2	Render	Yes	FastAPI, Flask, Django APIs
3	Railway	Small monthly free credit	AI backends, APIs, databases
4	Vercel	Hobby Plan	Next.js, React AI apps
5	Streamlit Community Cloud	Yes	Interactive ML dashboards
6	Google Colab + ngrok/Cloudflare Tunnel	Yes	Temporary AI demos
7	PythonAnywhere	Yes	Flask APIs and Python apps
8	Netlify	Yes	Static AI frontends
9	Cloudflare Workers	Yes	AI APIs, edge inference
10	Replicate	Free trial credits	Running open-source AI models

Top NLP Research Labs

Canada

Academic Institution	Lab Name & Link
Concordia University	Computational Linguistics at Concordia (CLAC)
Dalhousie University	Dalhousie Natural Language Processing and Data Mining Group (DNLP)
Université de Montréal	Recherche Appliquée en Linguistique Informatique (RALI)
University of British Columbia	Natural Language Processing Group
University of Ottawa	Natural Language Processing Lab
University of Toronto	Computational Linguistics
University of Waterloo	Natural Language Processing Lab
McGill University	Reasoning and Learning Lab (RLLAB)
Simon Fraser University	SFU Natural Language Laboratory (Natlang)
York University	Research Lab for Informational Retrieval and Knowledge Management

United States

Academic Institution	Lab Name & Link
Columbia University	NLP @ CU
MIT	MIT NLP Group
Monclair State University	Natural Language Processing Laboratory
Penn State University	PSU NLP LAB
Princeton University	Princeton NLP
Stanford University	The Stanford Natural Language Processing Group
University of California, Berkeley	Berkeley AI Research (BAIR)
University of California, Santa Barbara	UC Santa Barbara Natural Language Processing Group
University of Central Florida	UCF Natural Language Processing Group
University of Colorado, Boulder	Research Lab for Informational Retrieval and Knowledge Management
University of Delaware	NLP/AI Laboratory
University of Illinois Chicago	UIC NLP
University of Maryland	Computational Linguistics and Information Processing (CLIP)
University of North Carolina	Multimodal Understanding, Reasoning, and Generation for Language Lab (MURGe-Lab)



Top Leading Companies in the

Natural Language Processing Market

Baidu 百度

 **NVIDIA**

 intel

 Microsoft

Google

 IBM

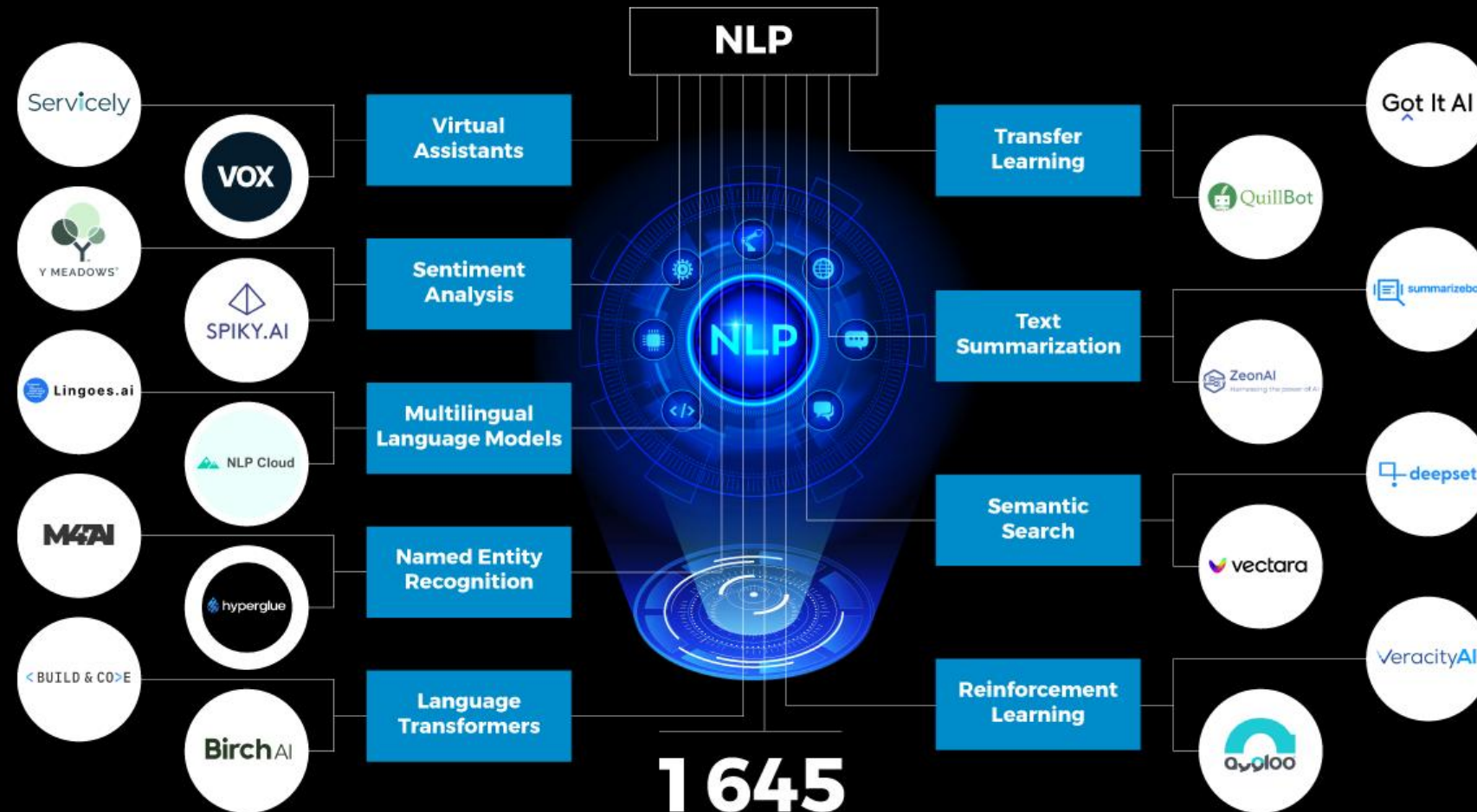
 **sas**

 genpact

 Adobe

VERINT

Top 9 Natural Language Processing Trends in 2023



Startups & emerging companies analyzed

NLP Engineer

Tasks

- Design and develop NLP systems
- Define appropriate datasets for language learning
- Use effective text representations to transform natural language into useful features
- Develop NLP systems according to requirements
- Train the developed model and run evaluation experiments
- Find and implement the right algorithms and tools for NLP tasks

Apple NLP Engineer Resume

Twila Goldston

NLP Engineer

As an experienced Natural Language Processing (NLP) Engineer with over 3 years of hands-on experience, I have developed strong expertise in the development and deployment of NLP solutions for a variety of industries. My key skills include designing, developing, and deploying NLP models and pipelines, as well as automating complex NLP tasks. I have successfully developed and implemented NLP solutions for a variety of tasks such as entity recognition and extraction, sentiment analysis, information retrieval, text summarization, and machine translation. I have also worked on developing custom NLP pipelines, utilizing deep learning models such as LSTM, Transformer, and BERT. Additionally, I have strong knowledge of various NLP libraries such as Spacy, NLTK, Gensim, and Scikit-learn, as well as programming languages such as Python and Java.

Throughout my career, I have been involved in a number of successful projects, such as developing a text summarization system for a news media company, an entity extraction system for a legal services company, and a sentiment analysis system for a customer service provider. I have also collaborated with data scientists, engineers, and product managers to ensure successful project execution.

I am confident that my experience and skills will be an asset to any organization.

twila.goldston@gmail.com
(333) 966-5934
Boulder, CO

Education
Master of Science in Natural Language Processing at University of Colorado Boulder
Aug 2016 - May 2020
Relevant Coursework: Algorithms for Natural Language Processing, Machine Learning for Natural Language Processing, Deep Learning for Natural Language Processing, Computational Semantics, Natural Language Understanding, and Natural Language Generation.

Links
[linkedin.com/in/twilagoldston](https://www.linkedin.com/in/twilagoldston)

Skills
Machine Learning
Natural Language Processing (NLP)
Data Analysis & Visualization

Employment History
Lead NLP Engineer at IBM, CO
Jan 2023 - Present

- Developed an automated Natural Language Processing (NLP) system for IBM CO, resulting in a 60% reduction in processing time for customer service requests.
- Led a team of 5 engineers to create a machine learning model for sentiment analysis, achieving an accuracy of 85%.
- Developed a new NLP-based chatbot that increased customer satisfaction by 15%, resulting in a 20% increase in sales.

Senior NLP Engineer at Google, CO
Aug 2020 - Nov 2022

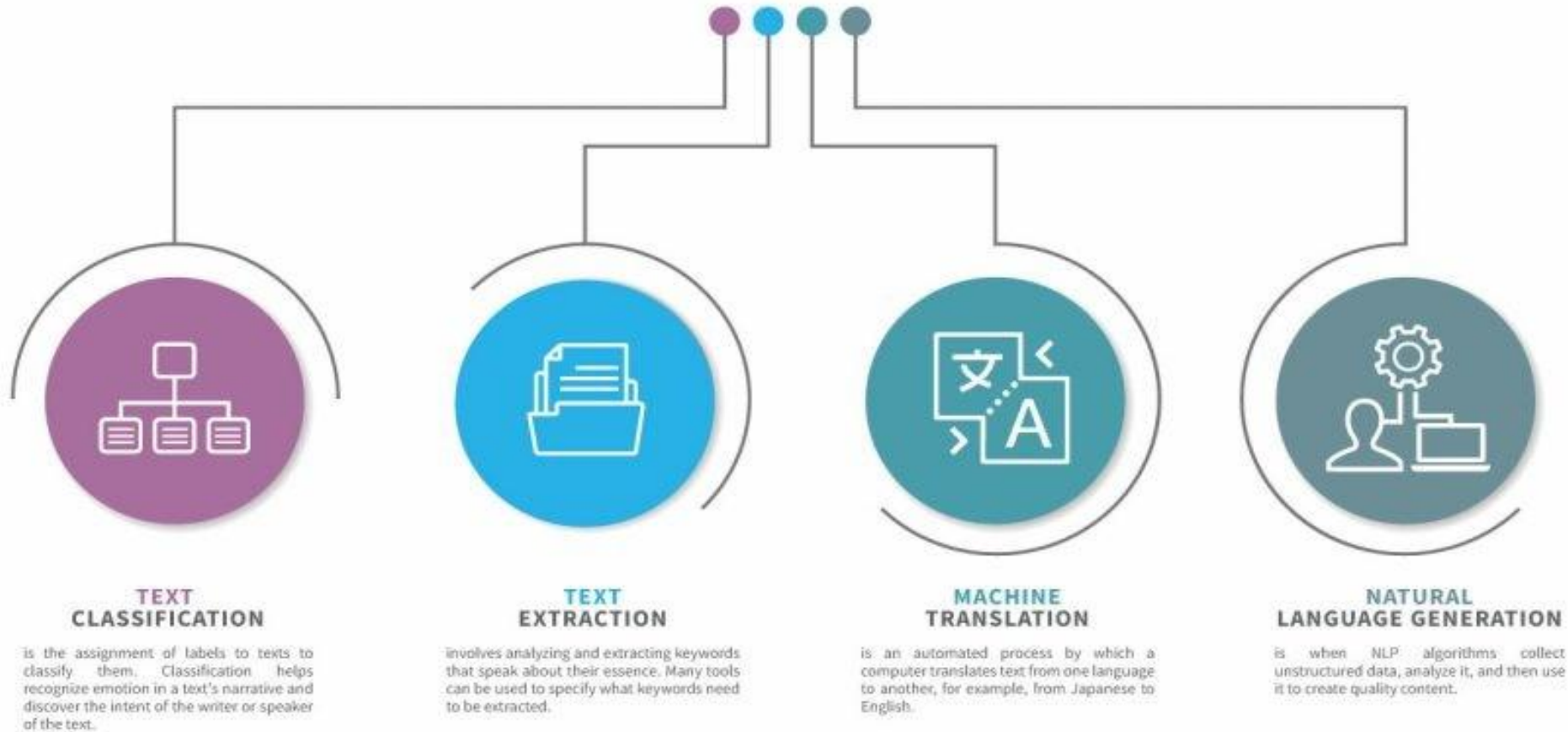
- Developed a neural network-based natural language processing (NLP) system for Google Search that improved user experience by 20%, resulting in a 10% increase in traffic to the site.
- Spearheaded the development of a conversational AI-based chatbot for Google Home that decreased customer service inquiries by 25%.
- Implemented an end-to-end NLP pipeline for Google Ads that increased its accuracy by 30%, resulting in 5% increase in ad revenue.

Certificates
Natural Language Processing Professional Certificate
Apr 2021
IBM Watson Natural Language Understanding Certification
Jun 2019

Memberships
Association for Computational Linguistics (ACL)
Natural Language Processing Special Interest Group (NLP-SIG)

WHAT IS NLP USED FOR

We can divide the functions of the NLP algorithm into four basic categories:



Unit 1 — What We'll Cover

7 hrs

01

Evolution of Natural Language

From human language origins to modern linguistics

02

Introduction to NLP

What NLP is, challenges, and key tasks

03

Applications & Phases of NLP

Opinion mining, dialog, discourse + pipeline

04

Data Preprocessing Techniques

Cleaning, normalization, stop-word removal

05

Tokenization & Embedding

Word/sentence tokens; dense vector representations

06

Stemming & POS Tagging

Root extraction + grammatical role labeling

07

Named Entity Recognition (NER)

Identifying persons, places, organizations

UNIT 1

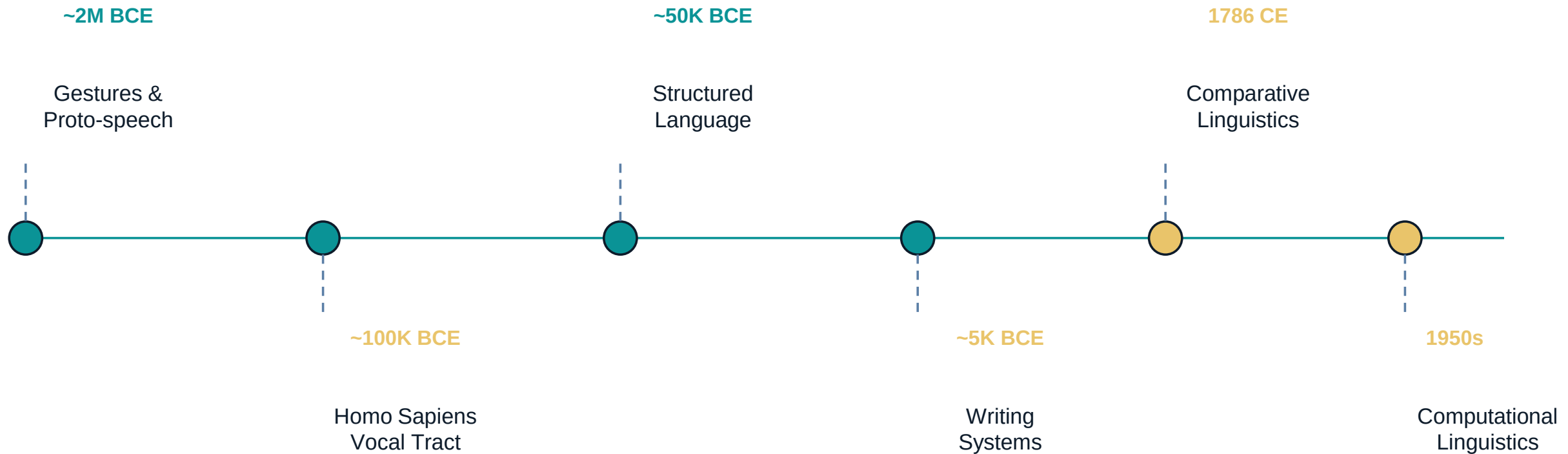
Evolution of Natural Language

From grunts to grammar — how human language developed

01

How Language Evolved

Topic 1



Key Insight: Language is uniquely human — recursive, symbolic, capable of expressing infinite ideas from finite elements (Chomsky, 1957).

UNIT 1

Introduction to NLP

Teaching computers to read, write, and understand human language

02

What is Natural Language Processing?

Topic 2

NLP is the branch of AI that gives computers the ability to understand, generate, and manipulate **human (natural) language** in both text and speech form.

Syntax

Structure of sentences
Grammar rules, parse trees

Semantics

Meaning of words and
sentences in context

Pragmatics

Intent & context beyond
literal meaning

Core Challenges of NLP

- ▶ Ambiguity: 'I saw the man with the telescope' — who had it?
- ▶ Context: Same word, different meanings (bank = riverbank or financial)
- ▶ Sarcasm & Irony: 'Oh great, another Monday!'
- ▶ Low-resource languages: billions of speakers, minimal digital data

Why Do We Need NLP? + Quick spaCy Demo

Topic 2

80%

*of all enterprise data is
unstructured text*



Chatbots & Dialog

ChatGPT, Alexa, customer support bots



Sentiment Analysis

Product reviews, Twitter, brand monitoring



Machine Translation

Google Translate, DeepL — 7000+ languages



Healthcare & Legal

Clinical notes, legal contract review

python — spaCy NLP pipeline demo

```
import spacy

# Load English model
nlp = spacy.load('en_core_web_sm')

text = "Apple was founded by Steve Jobs."
doc = nlp(text)

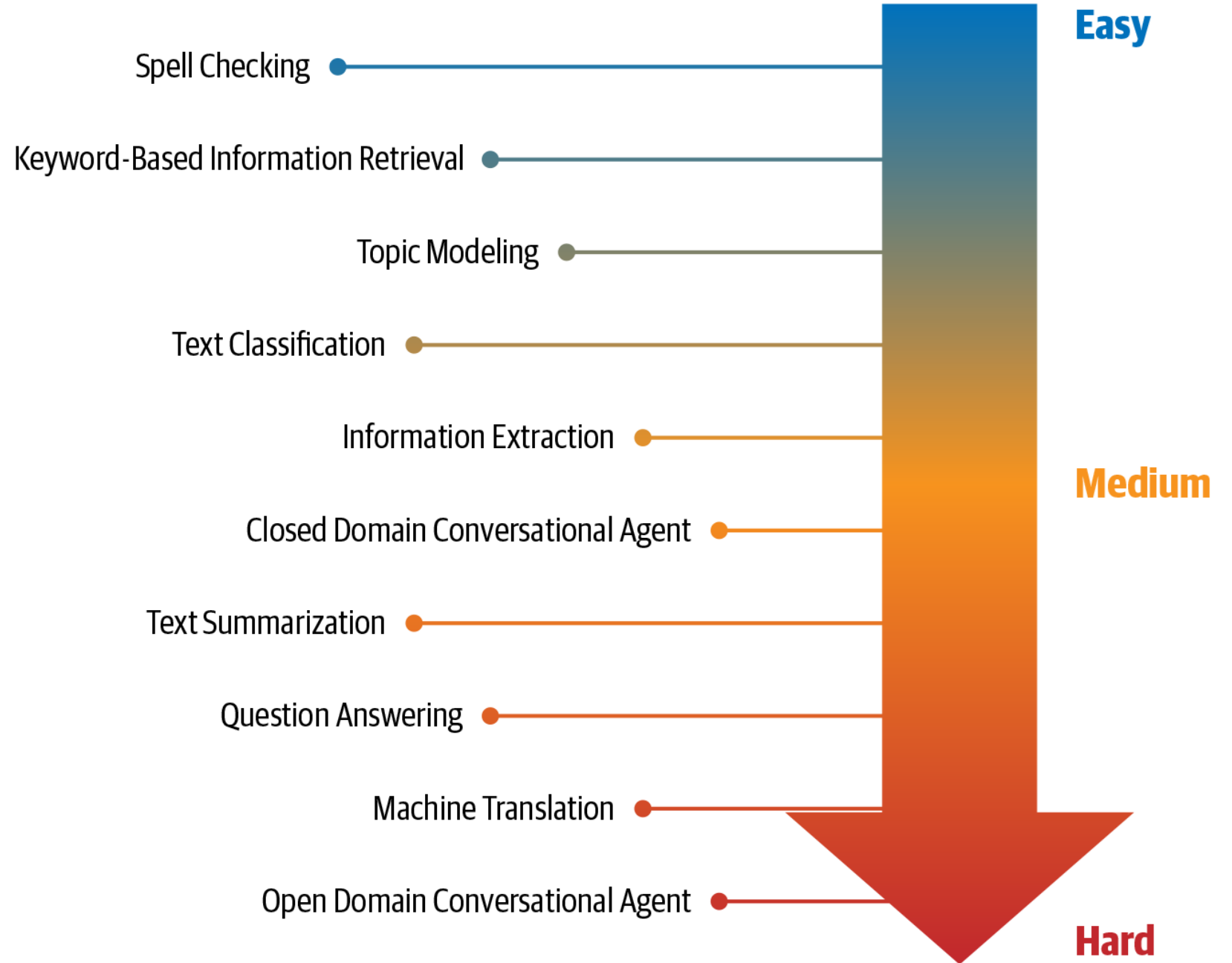
# Tokenization
for token in doc:
    print(token.text, token.pos_)

# Apple PROPN | was AUX | founded VERB

# Named Entity Recognition
for ent in doc.ents:
    print(ent.text, ent.label_)

# Apple ORG | Steve Jobs PERSON
```

NLP Tasks



Impact of Top 9 Natural Language Processing Trends

Virtual Assistants 18 %	Multilingual Language Models 15 %	Transfer Learning 8 %	Text Sum-marization 8 %
	Named Entity Recognition 11 %		
	Sentiment Analysis 18 %	Language Transformers 11 %	Semantic Search 6 %
		Reinforcement Learning 5 %	

Natural Language Processing (NLP)

Natural language processing

- The study of computational methods for understanding and generating human-readable text (or speech)

We mostly speak about text only in this course.

- The goal is to decode structured information from language, or to encode it in language.
- NLP is a subfield of AI, and one part of *computational linguistics*.



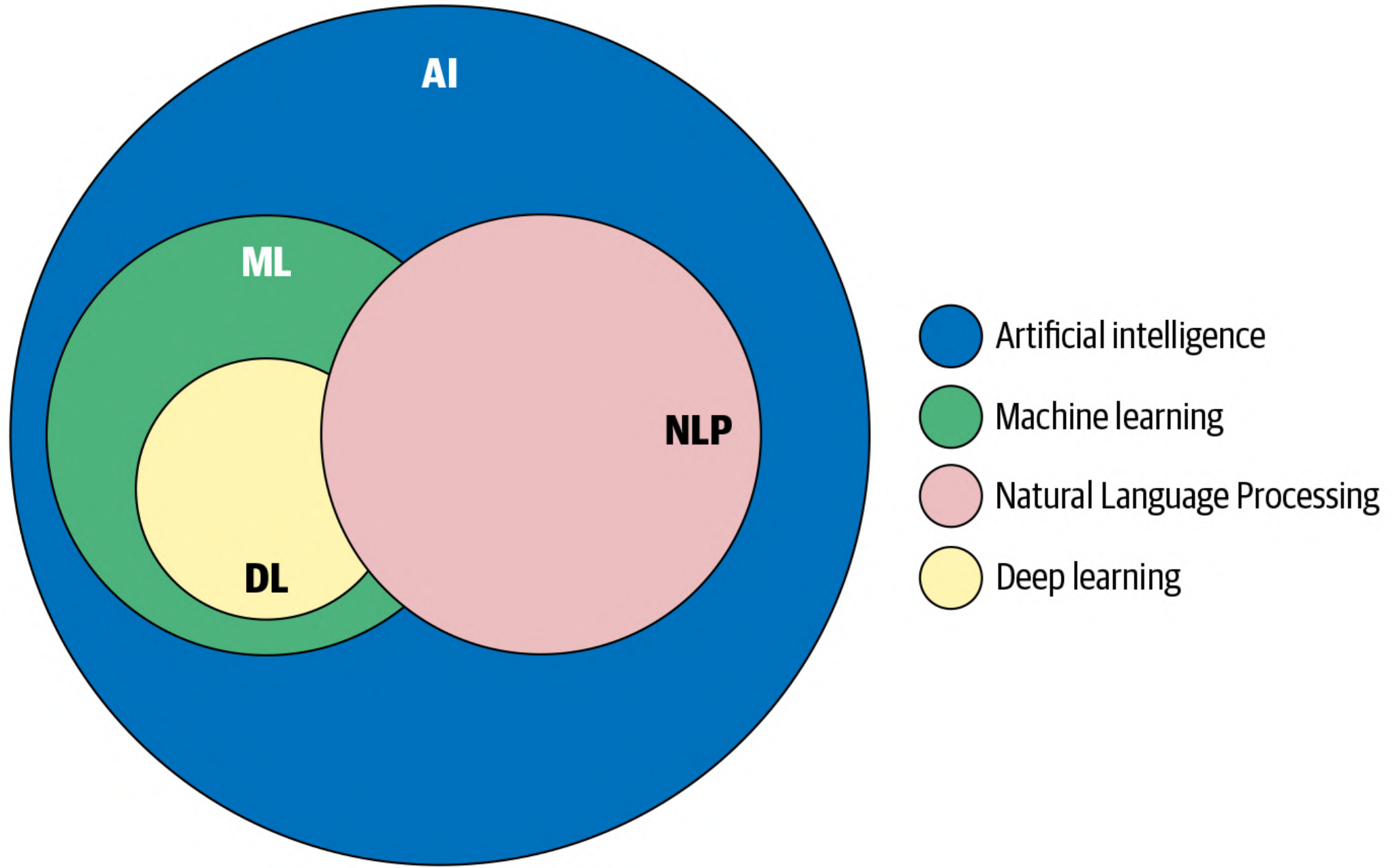
Computational linguistics

- Roughly, the intersection of computer science and linguistics
- **Methods** for tackling analysis and synthesis tasks from NLP
- **Models** to explain linguistic phenomena, using knowledge or statistics

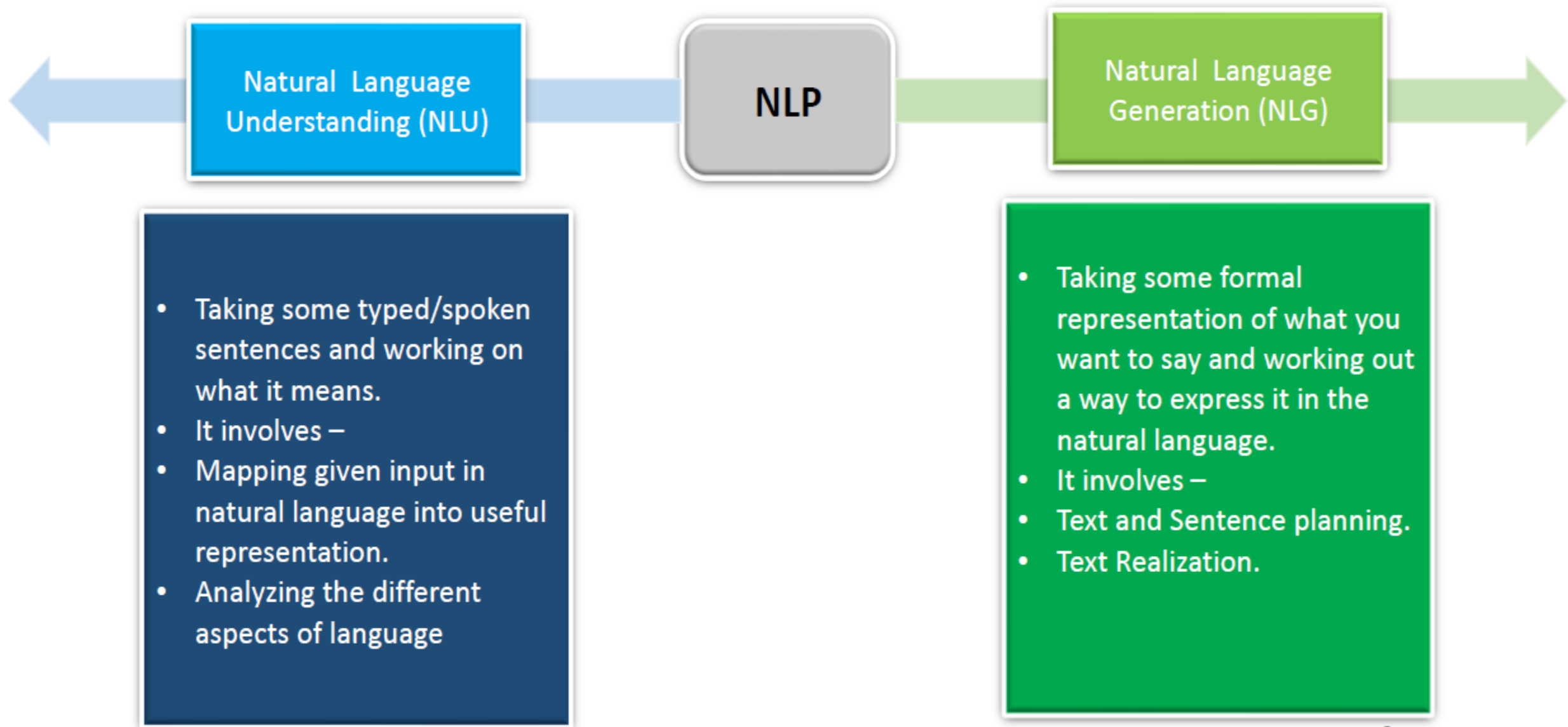
Linguistics

- The study of natural language(s) in terms of form, meaning, and context

Machine Learning, Deep Learning, and NLP



Types of NLP



Natural Language Processing (NLP)

Analysis and Synthesis

Types of NLP tasks

- **Analysis.** The decoding of structured information from text
 - **Synthesis.** The encoding of (structured) information into text
- Aka natural language understanding (NLU) and natural language generation (NLG)*

Selected analysis tasks

- Token and sentence splitting
- Stemming and lemmatization
- Part-of-speech tagging
- Constituency/Dependency parsing
- Named/Numeric entity recognition
- Reference resolution
- Entity/Temporal relation extraction
- Topic/Sentiment/Spam classification
- Text scoring/grading

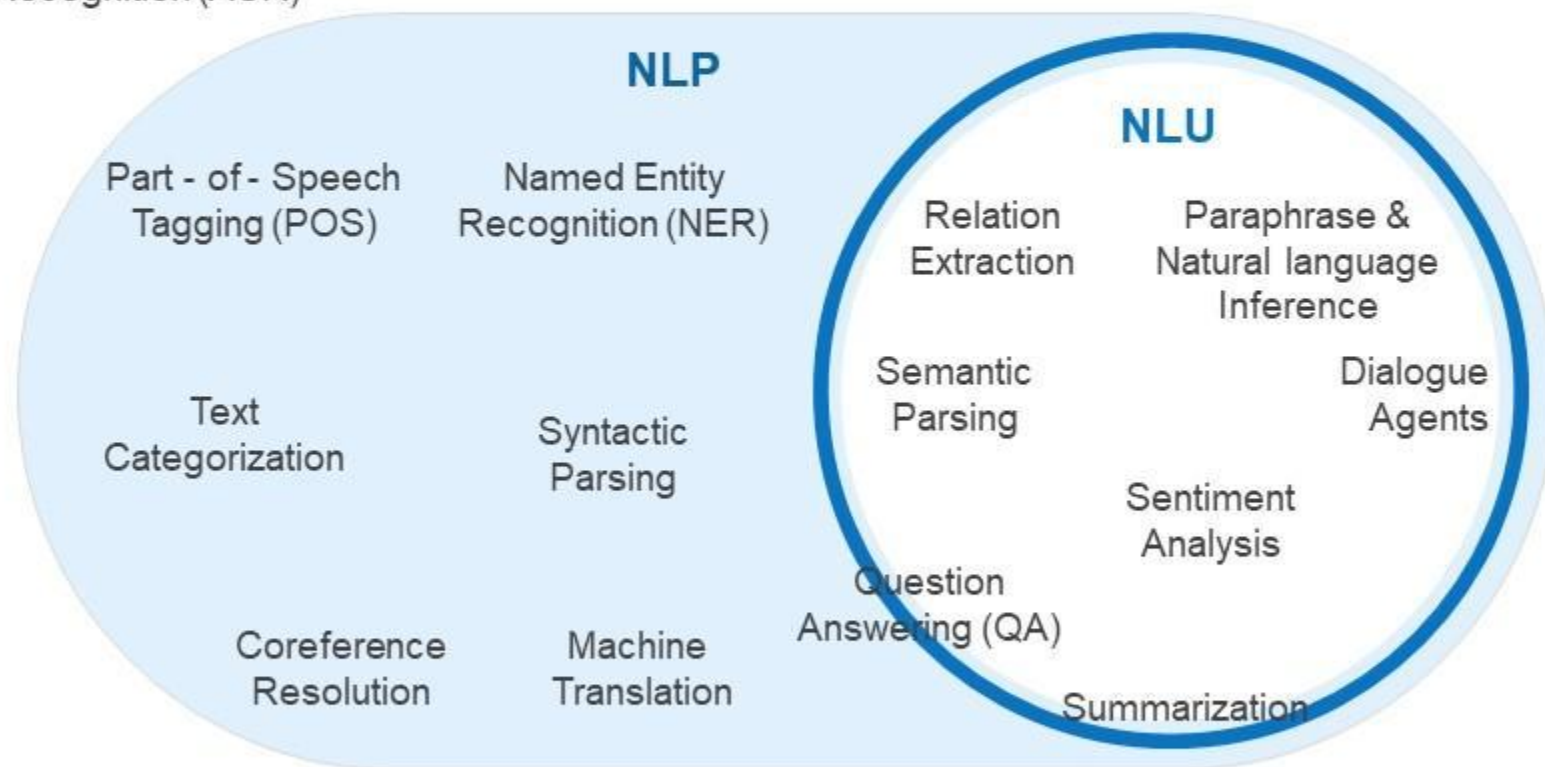
Selected synthesis tasks

- Lexicon creation
 - Free text generation
 - Sentence composition
 - Discourse composition
 - Spelling correction
 - Summarization
 - Text style transfer
 - Cluster labeling
- ... among many other tasks*

How is NLP related to NLU and NLG?

This slide represents how NLP relates to natural language understanding and natural language generation based on automatic speech recognition (ASR) and text-to-speech (TTS).

Automatic Speech
Recognition (ASR)



Text-to-Speech
(TTS)

Natural Language Understanding transforms voice or text into usable representations that can be analyzed

NLG picks the appropriate words, generating phrases and sentences based on an internal representation. Corrections must be arranged for information to be communicated effectively

Add text here

Add text here



NLG

- Discourse Generation
- Lexical Choice
- Sentence Planning/Generation
- Realization
- Document Structuring

NLP

- Named entity Recognition (NER)
- Part-of-Speech Tagging (POS)
- Syntactic Parsing
- Coreference Resolution
- Machine Translation

NLU

- Lexical Ambiguity/Analysis
- Sentiment Analysis
- Topic Classification
- Information Extraction
- Entity Detection
- Summarization
- Semantic Parsing
- Syntactical Analysis

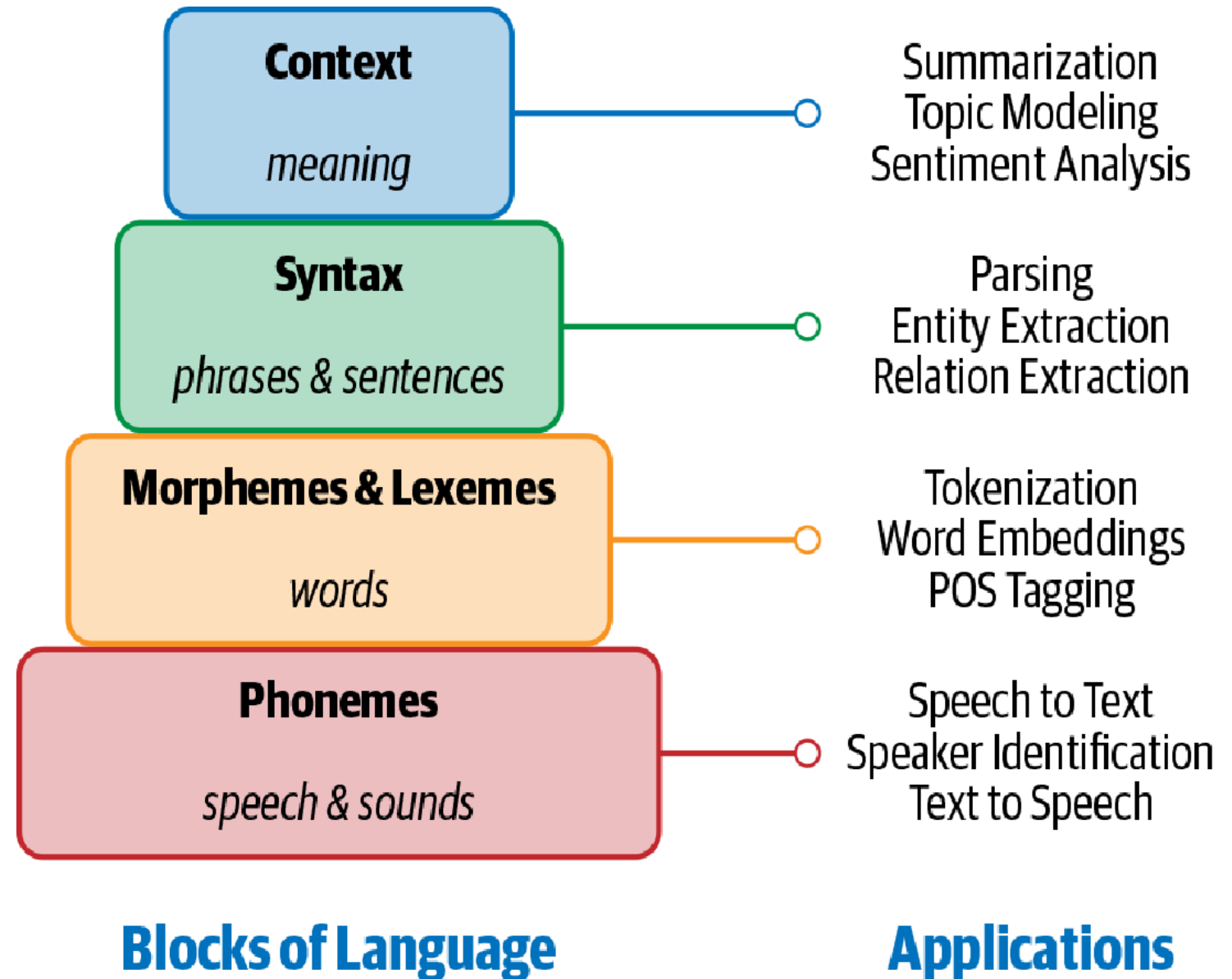
What is a language?

Language is a structured system of communication that involves complex combinations of its constituent components, such as characters, words, sentences, etc.

Linguistics is the systematic study of language. In order to study NLP, it is important to understand some concepts from linguistics about how language is structured

Lexeme refers to a single **word and all of its forms**. For example the word "go" in English has the forms "go" "goes" "went" and "going". All of these words are from the same lexeme "go."

Morpheme refers to the **smallest unit of meaning of a word** can be broken down into. For example the word "cats" This can be broken down into "cat-s" "cat" carries the meaning of the furry four legged animal and "-s" carries the meaning of plural.



Consonant phonemes, with sample words		Vowel phonemes, with sample words	
1. /b/ - bat	13. /s/ - sun	1. /a/ - ant	13. /oi/ - coin
2. /k/ - cat	14. /t/ - tap	2. /e/ - egg	14. /ar/ - farm
3. /d/ - dog	15. /v/ - van	3. /i/ - in	15. /or/ - for
4. /f/ - fan	16. /w/ - wig	4. /o/ - on	16. /ur/ - hurt
5. /g/ - go	17. /y/ - yes	5. /u/ - up	17. /air/ - fair
6. /h/ - hen	18. /z/ - zip	6. /ai/ - rain	18. /ear/ - dear
7. /j/ - jet	19. /sh/ - shop	7. /ee/ - feet	19. /ure/ ⁴ - sure
8. /l/ - leg	20. /ch/ - chip	8. /igh/ - night	20. /ə/ - corner (the 'schwa' - an unstressed vowel sound which is close to /u/)
9. /m/ - map	21. /th/ - thin	9. /oa/ - boat	
10. /n/ - net	22. /th/ - then	10. /oo/ - boot	
11. /p/ - pen	23. /ng/ - ring	11. /oo/ - look	
12. /r/ - rat	24. /zh/ ³ - vision	12. /ow/ - cow	

Figure 1-4. Phonemes and examples

unbreakable <i>un + break + able</i>	cats <i>cat + s</i>
tumbling <i>tumble + ing</i>	unreliability <i>un + rely + able + ity</i>

Figure 1-5. Morpheme examples

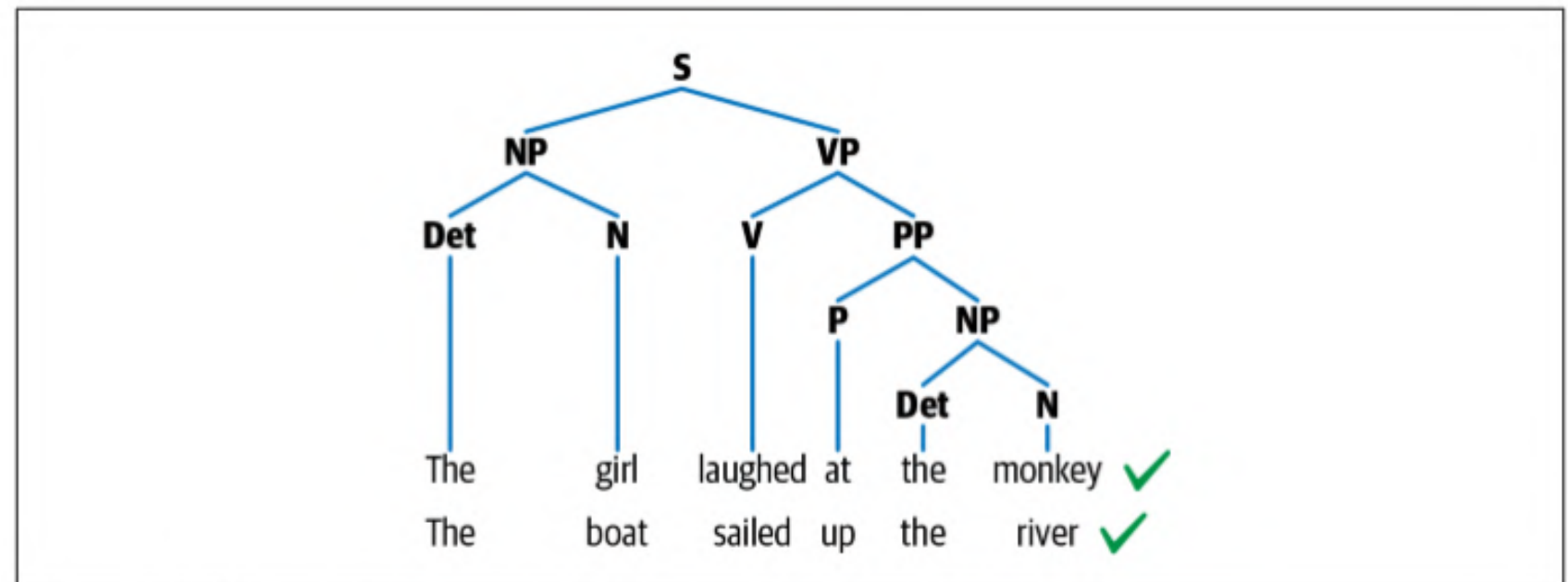


Figure 1-6. Syntactic structure of two syntactically similar sentences

NLP Terminology

- . **Phonology** – It is study of organizing sound systematically.
- . **Morphology**: The study of the formation and internal structure of words.
- . **Morpheme** – It is primitive unit of meaning in a language.
- . **Syntax**: The study of the formation and internal structure of sentences.
- . **Semantics**: The study of the meaning of sentences.
- . **Pragmatics** – It deals with using and understanding sentences in different situations and how the interpretation of the sentence is affected.
- . **Discourse** – It deals with how the immediately preceding sentence can affect the interpretation of the next sentence.
- . **World Knowledge** – It includes the general knowledge about the world.

Core NLP Language Levels

Easy Mnemonic

P -> M -> M -> S -> S -> P -> D -> W

Phonology = Sounds

Morphology = Word Formation

Morpheme = Meaningful Parts

Syntax = Sentence Structure

Semantics = Sentence Meaning

Pragmatics = Practical Meaning /
Context

Discourse = Sentence Connection

World Knowledge = Real-World
Understanding

Phonology

Speech sounds

Ex 1: Bat vs Cat

Ex 2: Ship vs Sheep

One sound change can change meaning.

Morphology

Word formation

Ex 1: un + happy + ness

Ex 2: play + ed

Words are built from smaller units.

Morpheme

Meaningful parts

Ex 1: cat + s

Ex 2: re + write

Each part contributes meaning.

Syntax

Sentence structure

Ex 1: The boy eats an apple.

Ex 2: She is reading a book.

Words follow grammar rules.

Semantics

Sentence meaning

Ex 1: The bank is closed.

Ex 2: I saw a bat.

Meaning depends on word sense.

Pragmatics

Practical/context meaning

Ex 1: Can you open the window?

Ex 2: It is cold here.

Context reveals intended meaning.

Discourse

Sentence connection

Ex 1: Riya bought a laptop. It is fast.

Ex 2: John dropped the glass. It broke.

Later sentences depend on earlier ones.

World Knowledge

Real-world understanding

Ex 1: The glass fell and broke.

Ex 2: Birds usually fly.

Understanding uses common knowledge.

Memory Trick: start from sounds, build words, arrange sentences, understand meaning, use context, connect ideas, and apply real-world knowledge.

Sounds

>

Words

>

Parts

>

Grammar

>

Meaning

>

Context

>

Connection

>

World

Applications & Phases of NLP

Where NLP is used and how a system processes text step by step

03



Chatbots & Dialog

ChatGPT, Alexa, Google Assistant



Speech Recognition

Siri, Cortana, YouTube CC



Machine Translation

Google Translate, DeepL



Opinion Mining

Product reviews, Twitter analysis



Text Summarization

News snippets, research abstracts



Information Retrieval

Search engines, Q&A systems

Applications of NLP



Applications of NLP

Core Tasks

*Covered in
Chapters 3–7*



Text
Classification



Information
Extraction



Conversational
Agent



Information
Retrieval



Question
Answering Systems

General Applications

*Covered in
Chapters 4–7*



Spam
Classification



Calendar Event
Extraction



Personal
Assistants



Search
Engines

JEOPARDY!

Jeopardy!

Industry Specific

*Covered in
Chapters 8–10*



Social Media
Analysis



Retail Catalog
Extraction



Health Records
Analysis



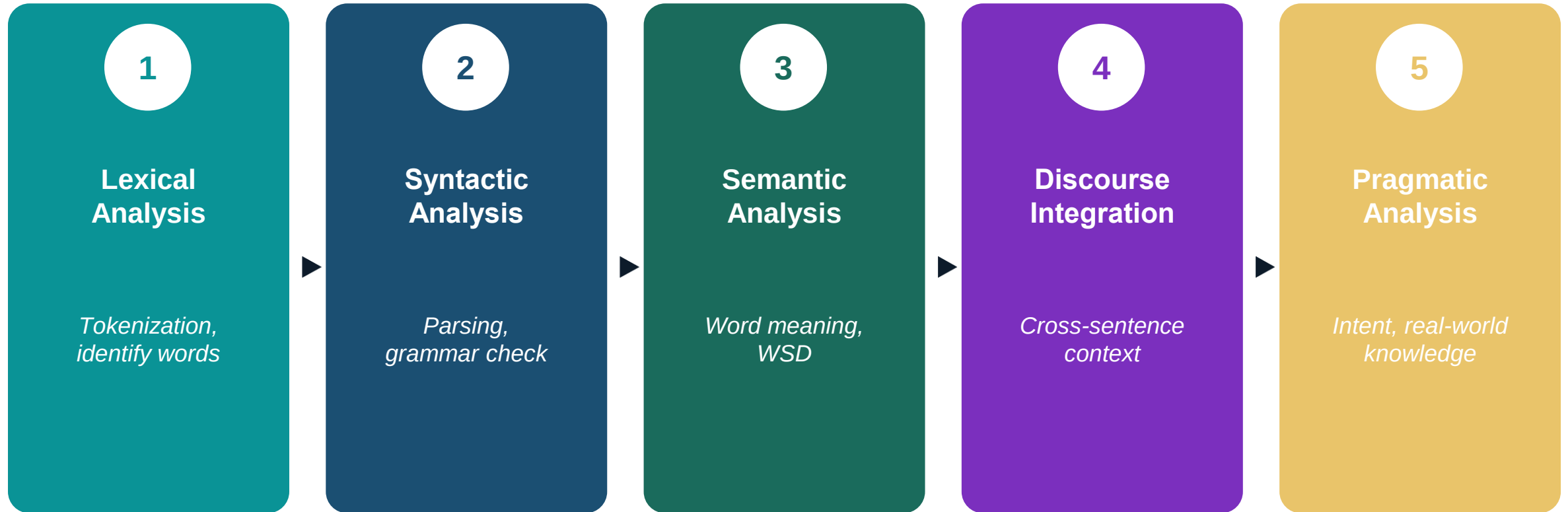
Financial
Analysis



Legal Entity
Extraction

Phases of the NLP Pipeline

Topic 3



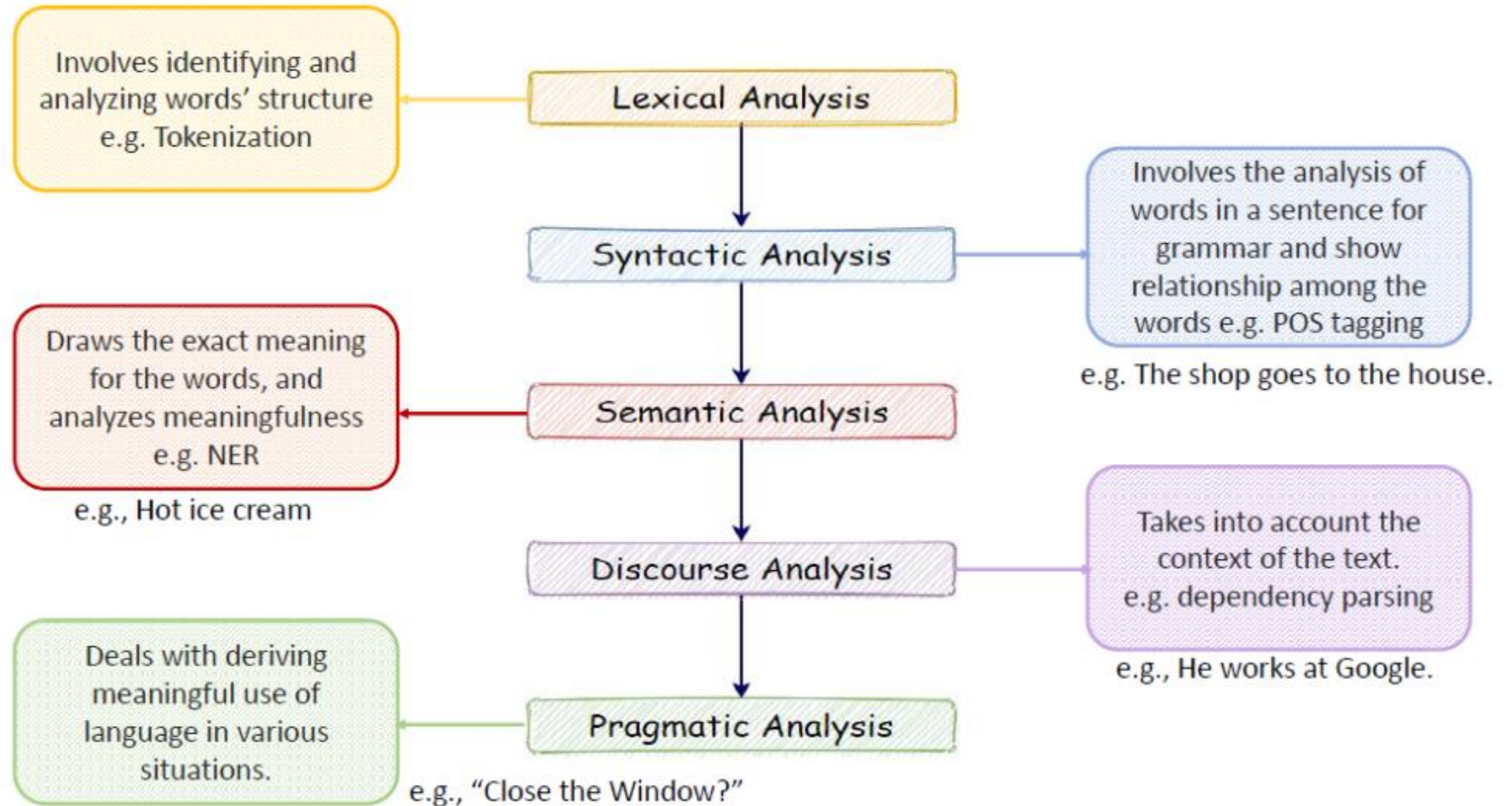
Input: Raw Text

WSD: Word Sense Disambiguation

Output: Structured Knowledge / Action

Example → "Apple acquired the startup." Lexical: tokens | Syntactic: NP VP NP | Semantic: acquisition event | Pragmatic: business context

Phases in NLP



Lexical Analysis –

- The first phase of NLP is the Lexical Analysis.
- This phase scans the source code as a stream of characters and converts it into meaningful lexemes.
- It divides the whole text into paragraphs, sentences, and words.

Syntactic Analysis (Parsing) –

- Syntactic Analysis is used to check grammar, word arrangements, and shows the relationship among the words.
- The sentence such as “The school goes to boy” is rejected by English syntactic analyzer.

Semantic Analysis –

- Semantic analysis is concerned with the meaning representation.
- It mainly focuses on the literal meaning of words, phrases, and sentences.
- The semantic analyzer disregards sentence such as “hot ice-cream”.

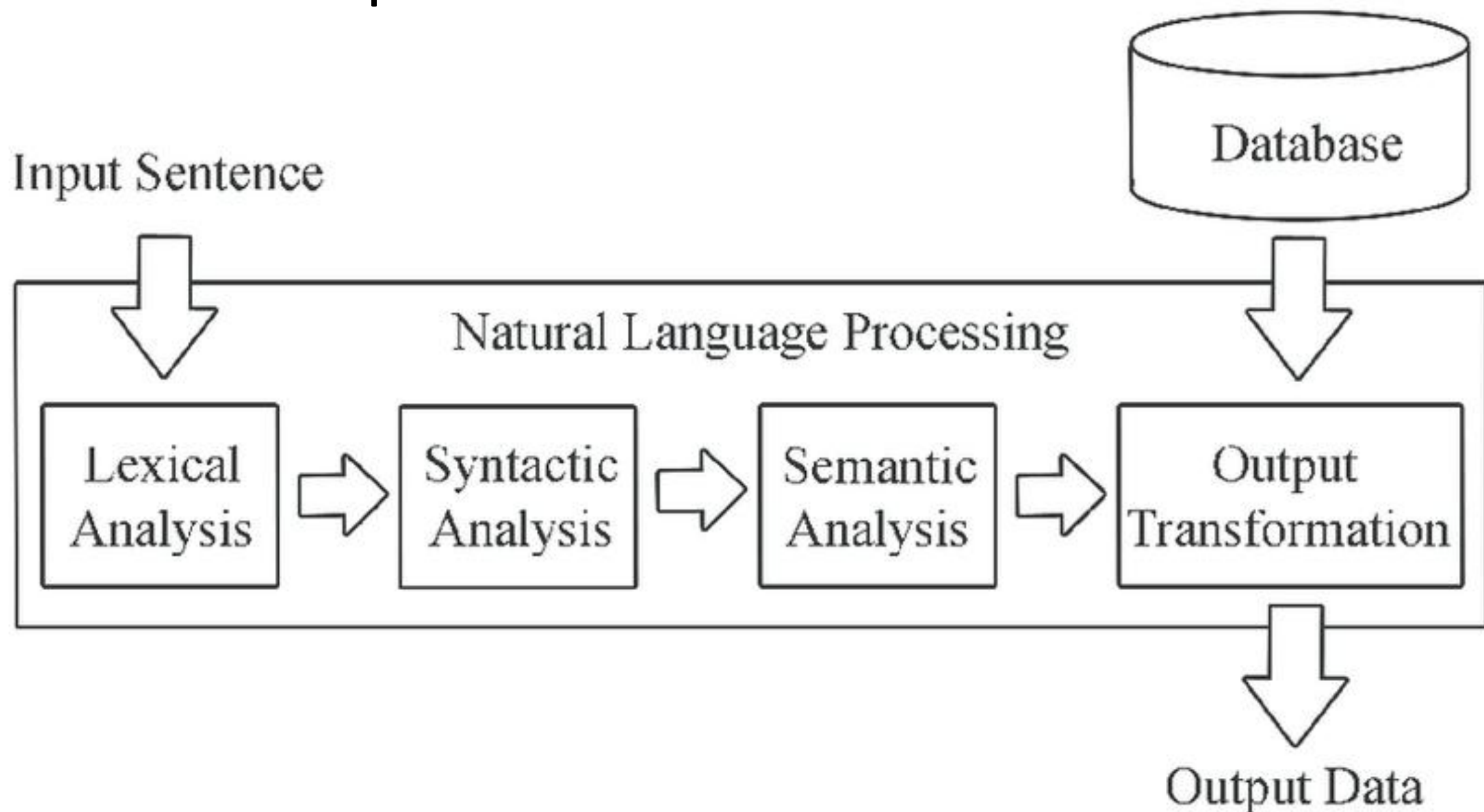
Discourse Integration –

- Discourse Integration depends upon the sentences that proceeds it and also invokes the meaning of the sentences that follow it.

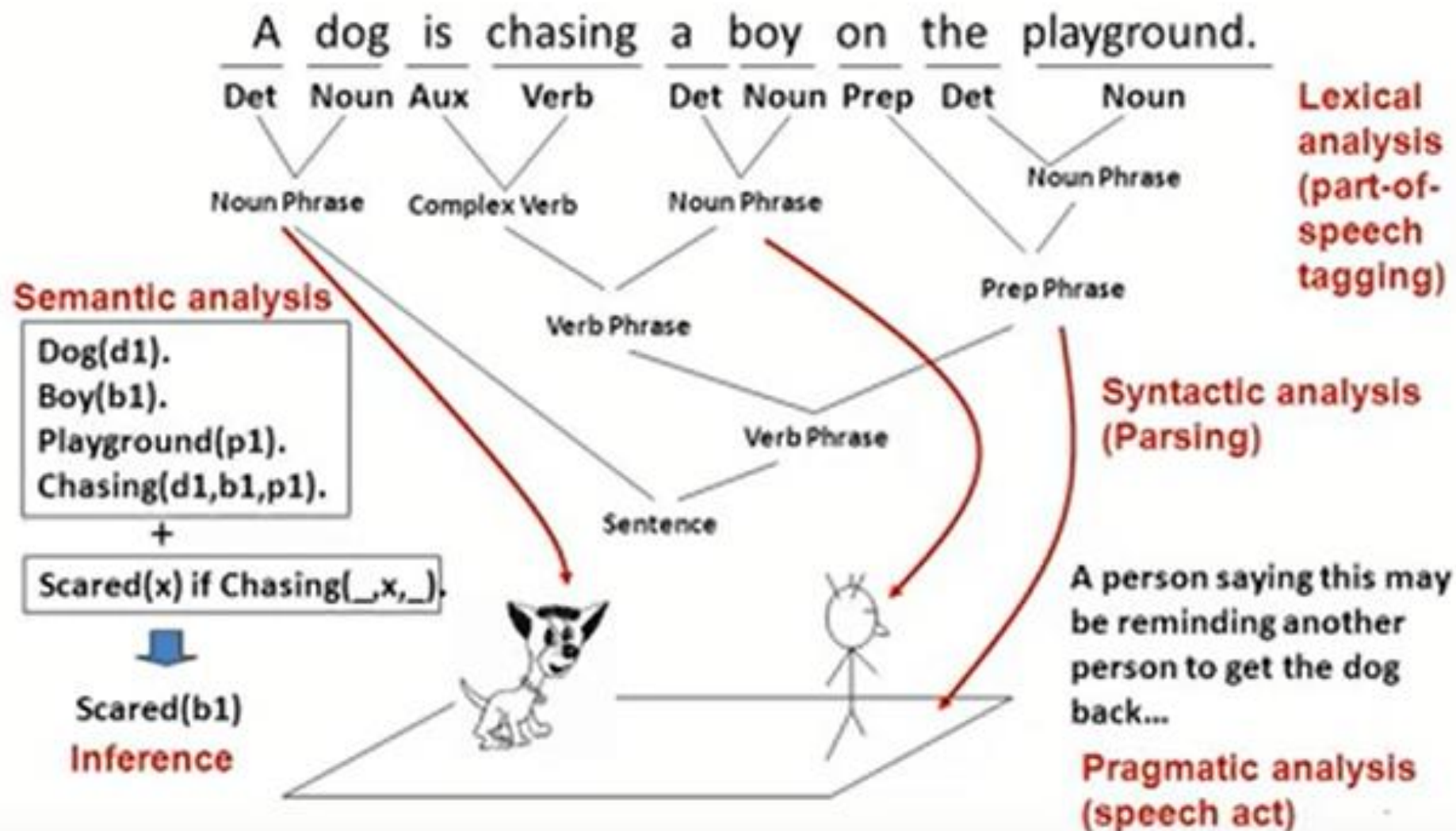
Pragmatic Analysis –

- During this, what was said is re-interpreted on what it actually meant.
- It involves deriving those aspects of language which require real world knowledge.
- **Example:** "Open the door" is interpreted as a request instead of an order.

Phases in NLP: Pipeline

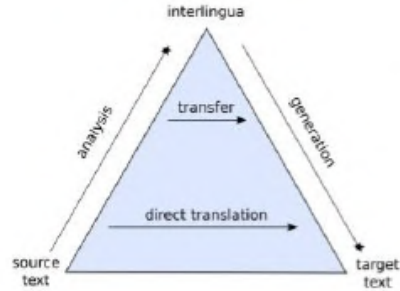


Phases in NLP: Example

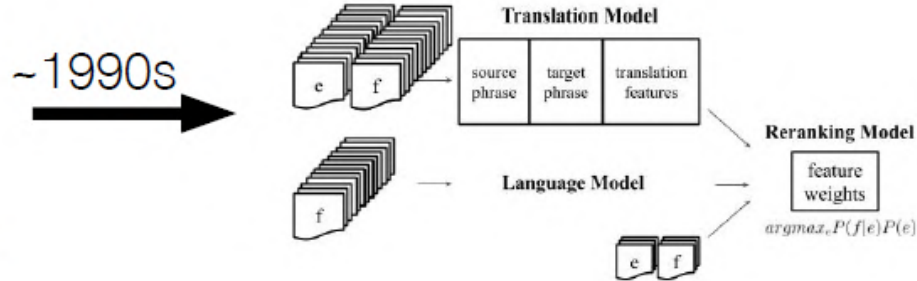


Symbolic and probabilistic NLP

Logic/Rule-based NLP

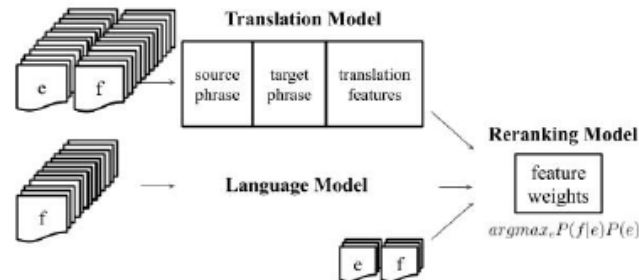


Statistical NLP

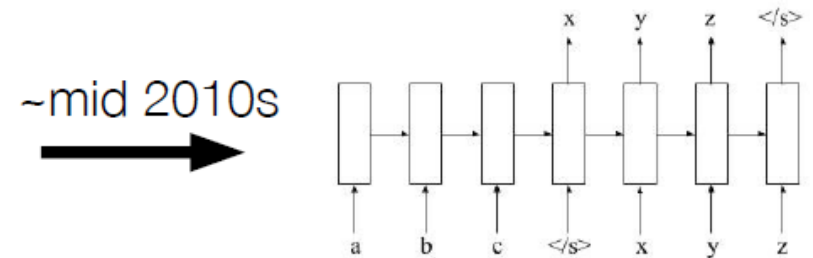


Probabilistic and Connectionist NLP

Engineered features



Learned features



Challenges in NLP

- **Contextual words and phrases**

The same words and phrases can have different meanings according the context of a sentence and many words – especially in English – have the exact same pronunciation but totally different meanings.

For example:

*I **ran** to the store because we **ran** out of milk.*

*Can I **run** something past you real quick?*

*The house is looking really **run** down.*

Ambiguity

Ambiguity in NLP refers to sentences and phrases that potentially have two or more possible interpretations.

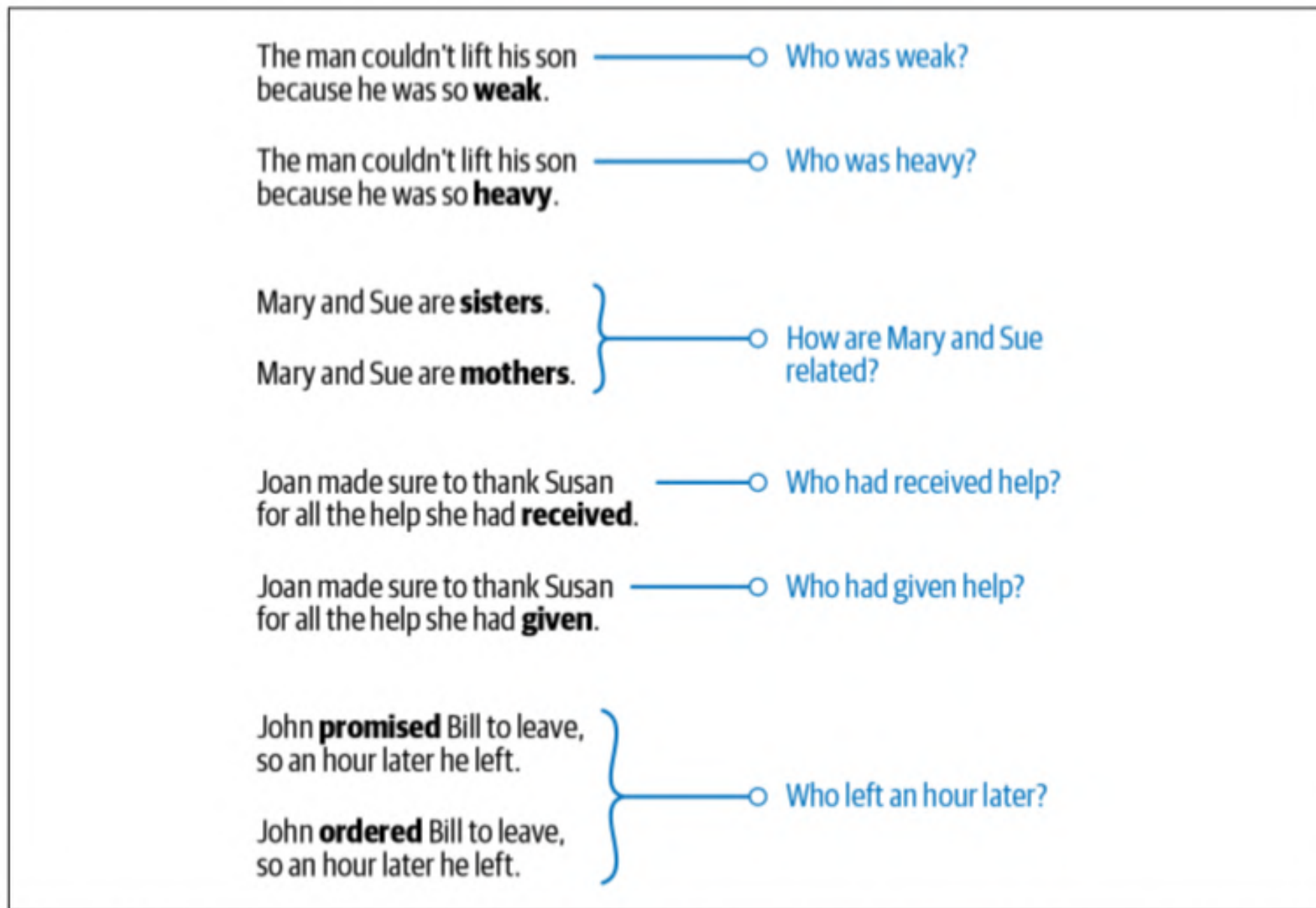


Figure 1-7. Examples of ambiguity in language from the Winograd Schema Challenge

Ambiguity

- Ambiguity at multiple levels:
 - Word sense: **bank** (finance or river)
 - Part of speech: **chair** (noun or verb?)
 - Syntactic structure: **I saw the man with the telescope**
 - Multiple: **I saw her duck**



Types of Ambiguity

Lexical ambiguity

A word that could be used as a verb, noun, or adjective.

Semantic ambiguity

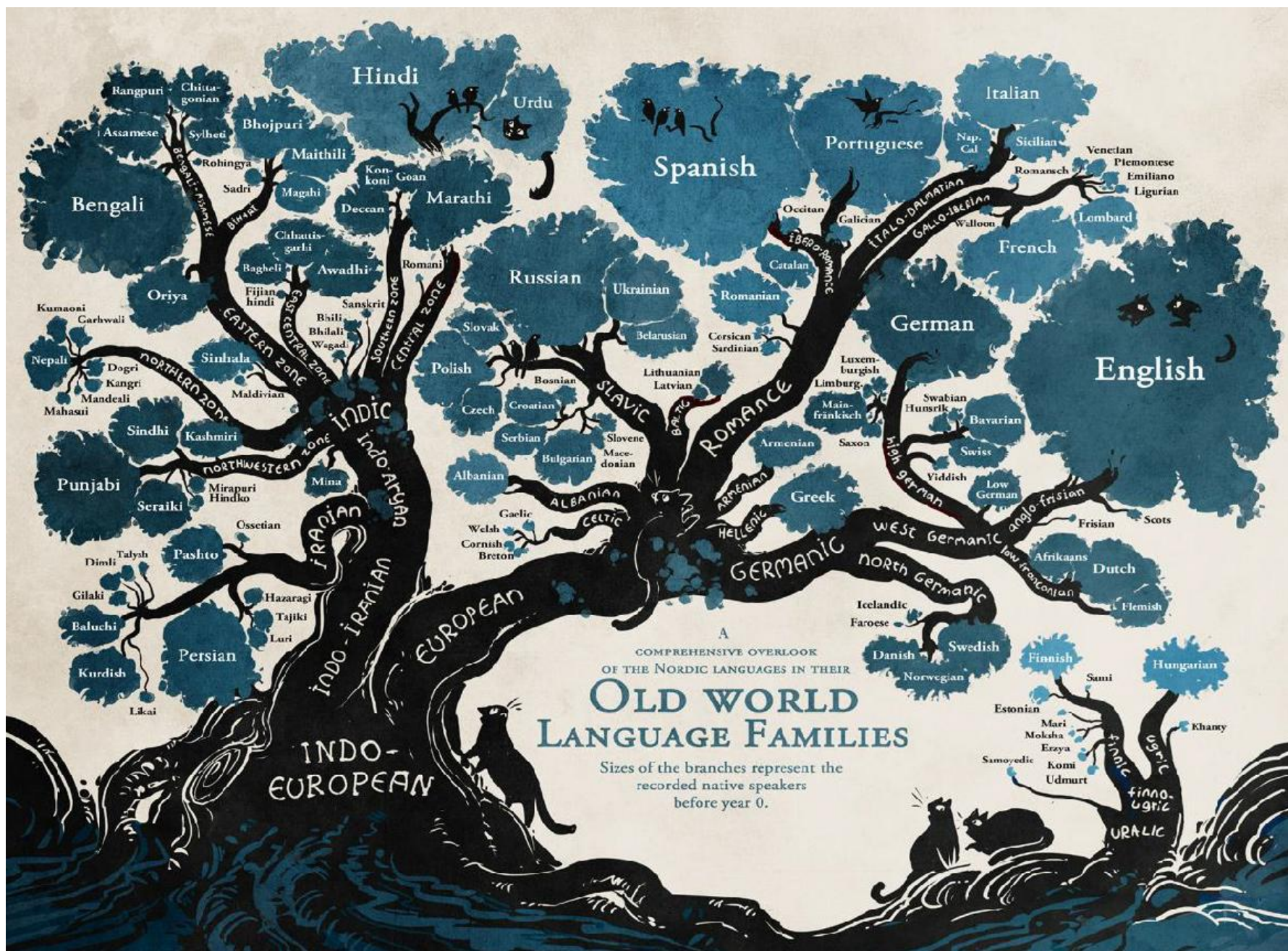
The interpretation of a sentence in context. E.g. *“I saw the boy on the beach with my binoculars”* — I saw the boy through my binoculars, or the boy had my binoculars with him.

Syntactic ambiguity

Structure creates the confusion of meaning — *“with my binoculars”* could modify the verb “saw” or the noun “boy”.

*Even for humans this sentence is difficult to interpret without surrounding context. **POS (part-of-speech) tagging** is one NLP solution that can help solve the problem, somewhat.*

Non Standardization in Languages



Challenges

Practical Issues

Common practical issues

- NLP faces effectiveness, efficiency, and robustness issues in practice.
- How to deal with such issues will be discussed at the end of this course.

Effectiveness issues

- **Effectiveness.** The extent to which the output of a method is correct
- Methods may not be effective enough for use in real-life applications.

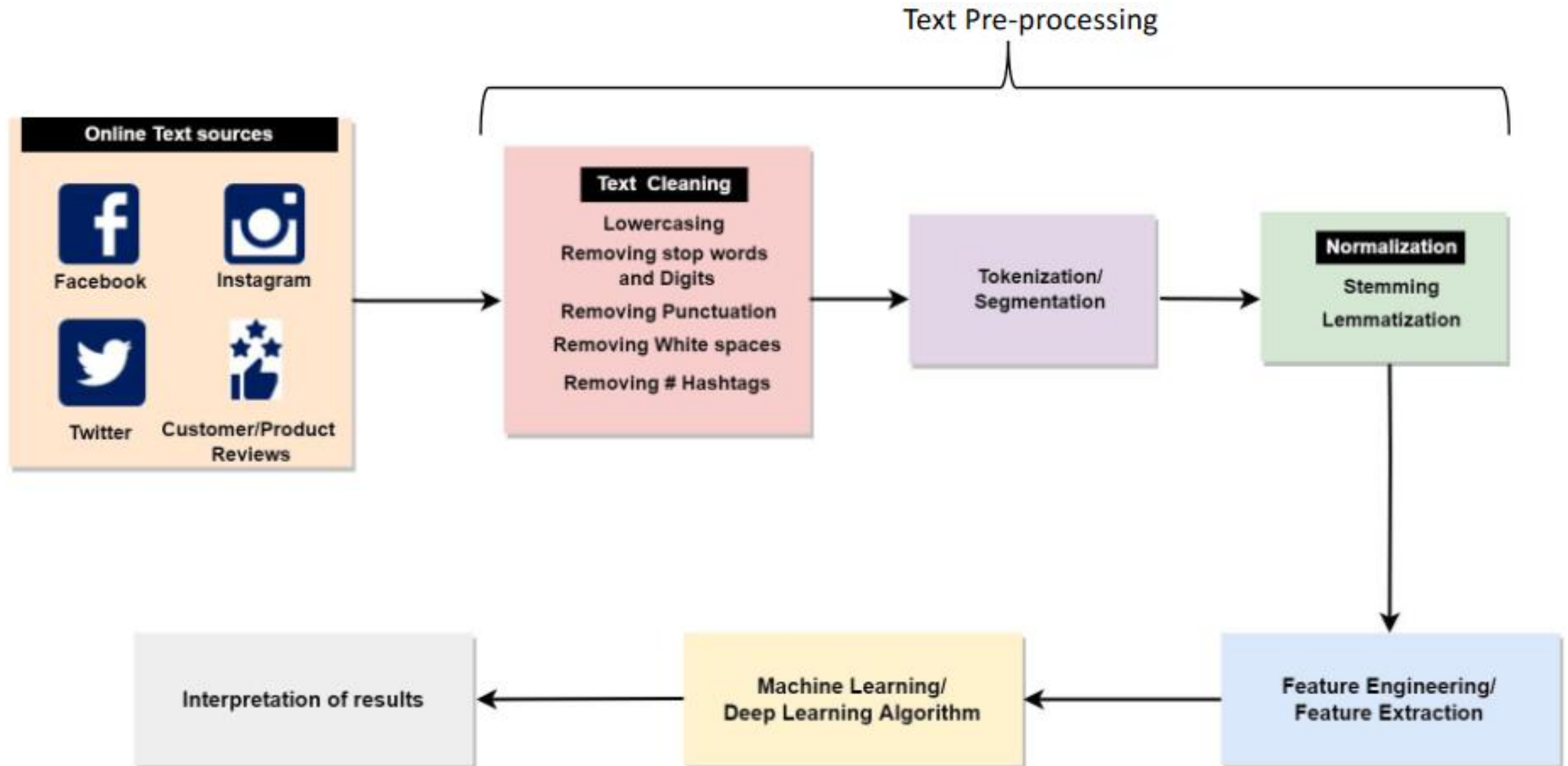
Efficiency issues

- **Efficiency.** The run-time, space, or energy consumption of a method
- Methods may not be efficient enough when applied to big text amounts.

Robustness issues

- **Robustness.** The effectiveness of a method across domains of text
- Methods may not be robust enough on data different from training data.

NLP implementation Pipeline



Text Preprocessing & Feature Engineering

Text Preprocessing

Text Cleaning — HTML tag removal, emoji handling, spelling checker, etc.

Basic Preprocessing — tokenization, stop-word removal, removing digits, lowercasing, removing punctuation, hashtags and special characters.

Digression — POS, NER.

Normalization — stemming and lemmatization.

Vectorization — one-hot encoding, BoW, GloVe, TF-IDF.

Feature Engineering

- ◆ One-Hot Encoder
- ◆ Bag of Words (BoW)
- ◆ n-grams
- ◆ TF-IDF
- ◆ Word2Vec
- ◆ GloVe

Model Building

Heuristics Based

- ◆ Lexicon based
- ◆ Regex based

ML Based

- ◆ Naïve Bayes
- ◆ SVM
- ◆ Decision Trees

DL Based

- ◆ CNN
- ◆ RNN
- ◆ Transformers
- ◆ Autoencoders

Input

Chaplin wrote, directed, and composed the music for most of his films.

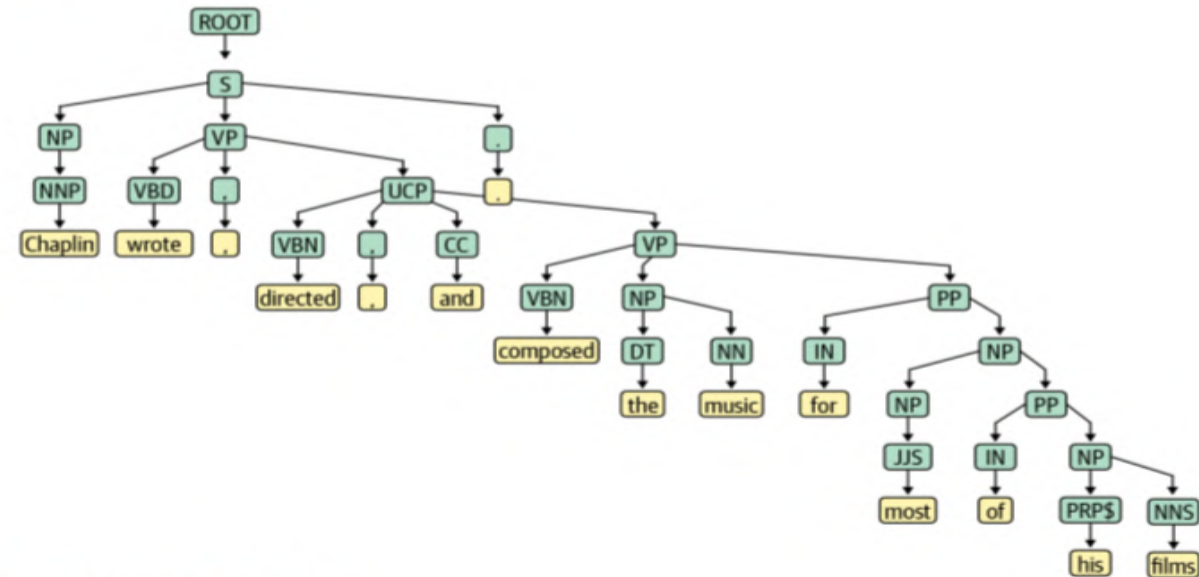
Tokenization with Lemmatization

Chaplin wrote, directed, and composed the music for most of his films.

POS Tagging

Chaplin wrote, directed, and composed the music for most of his films.

Parse Tree



Coreference Resolution

Mention ----- coref ----- Mention
Chaplin wrote, directed, and composed the music for most of his films.

Figure 2-10. Output from different stages of NLP pipeline processing

UNIT 1

Data Preprocessing Techniques

Cleaning and normalizing raw text before modeling

04

Text Preprocessing — Steps & Python Code

Topic 4

1. Lowercasing

"NLP" → "nlp"

2. Remove Punctuation

"Hello, world!" → "Hello world"

3. Remove Stop Words

"the cat sat on mat" → "cat sat mat"

4. Remove HTML/Noise

"<p>Buy now!</p>" → "Buy now"

5. Normalize Numbers

"I have 3 cats" → "I have NUM cats"

```
python — NLTK preprocessing pipeline

import re, nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

def preprocess(text):
    text = text.lower()
    text = re.sub(r'^\w\s', '', text)
    text = re.sub(r'\d+', 'NUM', text)
    stop = set(stopwords.words('english'))
    tokens = word_tokenize(text)
    return [w for w in tokens
            if w not in stop]

# Test
preprocess("The Cat SAT on 3 Mats!!")
# ['cat', 'sat', 'NUM', 'mat']
```

Punctuation Removal

- In this step, all the punctuations from the text are removed. string library of Python contains some pre-defined list of punctuations such as `'!"#$%&'()*+,-./:;?@[\\]^_`{|}~'`

v2	clean_msg
Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...	Go until jurong point crazy Available only in bugis n great world la e buffet Cine there got amore wat
Ok lar... Joking wif u oni...	Ok lar Joking wif u oni
Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's	Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005 Text FA to 87121 to receive entry questionstd txt rateTCs apply 08452810075over18s
U dun say so early hor... U c already then say...	U dun say so early hor U c already then say
Nah I don't think he goes to usf, he lives around here though	Nah I dont think he goes to usf he lives around here though

Lowering the Text

- It is one of the most common text preprocessing Python steps where the text is converted into the same case preferably lower case. But it is not necessary to do this step every time you are working on an NLP problem as for some problems lower casing can lead to loss of information.
- For example, if in any project we are dealing with the emotions of a person, then the words written in upper cases can be a sign of frustration or excitement.

clean_msg	msg_lower
Go until jurong point crazy Available only in bugis n great world la e buffet Cine there got amore wat	go until jurong point crazy available only in bugis n great world la e buffet cine there got amore wat
Ok lar Joking wif u oni	ok lar joking wif u oni
Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005 Text FA to 87121 to receive entry questionstd txt rateTCs apply 08452810075over18s	free entry in 2 a wkly comp to win fa cup final tkts 21st may 2005 text fa to 87121 to receive entry questionstd txt ratetcs apply 08452810075over18s
U dun say so early hor U c already then say	u dun say so early hor u c already then say
Nah I dont think he goes to usf he lives around here though	nah i dont think he goes to usf he lives around here though

Stop Word Removal

- Stopwords are the commonly used words and are removed from the text as they do not add any value to the analysis. These words carry less or no meaning.
- NLTK library consists of a list of words that are considered stopwords for the English language. Some of them are : [i, me, my, myself, we, our, ours, ourselves, you, you're, you've, you'll, you'd, your, yours, yourself, yourselves, he, most, other, some, such, no, nor, not, only, own, same, so, then, too, very, s, t, can, will, just, don, don't, should, should've, now, d, ll, m, o, re, ve, y, ain, aren't, could, couldn't, didn't, didn't]

msg_tokenied	no_stopwords
[go, until, jurong, point, crazy, available, only, in, bugis, n, great, world, la, e, buffet, cine, there, got, amore, wat]	[go, jurong, point, crazy, available, bugis, n, great, world, la, e, buffet, cine, got, amore, wat]
[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]
[free, entry, in, 2, a, wkly, comp, to, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, to, 87121, to, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]	[free, entry, 2, wkly, comp, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, 87121, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]
[u, dun, say, so, early, hor, u, c, already, then, say]	[u, dun, say, early, hor, u, c, already, say]
[nah, i, dont, think, he, goes, to, usf, he, lives, around, here, though]	[nah, dont, think, goes, usf, lives, around, though]

Decontracting the words

- In English contractions, we often drop the vowels from a word to form the contractions. Removing contractions contributes to text standardization and is useful when we are working on Twitter data, on reviews of a product as the words play an important role in sentiment analysis.

```
def decontracted(phrase):  
    phrase = re.sub(r"won't", "will not", phrase)  
    phrase = re.sub(r"can't", "can not", phrase)  
    phrase = re.sub(r"n't", " not", phrase)  
    phrase = re.sub(r"\ 're", " are", phrase)  
    phrase = re.sub(r"\ 's", " is", phrase)  
    phrase = re.sub(r"\ 'd", " would", phrase)  
    phrase = re.sub(r"\ 'll", " will", phrase)  
    phrase = re.sub(r"\ 't", " not", phrase)  
    phrase = re.sub(r"\ 've", " have", phrase)  
    phrase = re.sub(r"\ 'm", " am", phrase)  
    return phrase
```

Text Normalization — RegEx & Contractions

Topic 4

Normalization Goals

Expand Contractions

don't → *do not*, *I'm* → *I am*

Standardize Spellings

colour → *color*, *normalise* → *normalize*

Handle URLs/Emails

https://sit.edu → *URL_TOKEN*

Remove Accents

café → *cafe*, *naïve* → *naive*

Whitespace Cleanup

'hello world' → *'hello world'*

 *python — normalization with regex & contractions*

```
import re
import unicodedata

CONTRACTIONS = {
    "don't": "do not",
    "i'm" : "i am",
    "won't": "will not"}

def normalize(text):
    text = text.lower()
    for k,v in CONTRACTIONS.items():
        text = text.replace(k, v)
    text = re.sub(r'https?:\/\/\S+', 'URL', text)
    return text

normalize("I don't visit https://sit.edu")
# → "i do not visit URL"
```

Natural language processing

Applications

- Machine Translation
- Information Retrieval
- Question Answering
- Dialogue Systems
- Information Extraction
- Summarization
- Sentiment Analysis

Core technologies

- Language modeling
- Part-of-speech tagging
- Syntactic parsing
- Named-entity recognition
- Coreference resolution
- Word sense disambiguation
- Semantic role labelling

Tokenization & Embedding

Breaking text into units and representing them as numbers

05

Tokenization

- Tokenization is essentially splitting a phrase, sentence, paragraph, or an entire text document into smaller units, such as individual words or terms. Each of these smaller units are called tokens.

msg_lower	msg_tokenied
go until jurong point crazy available only in bugis n great world la e buffet cine there got amore wat	[go, until, jurong, point, crazy, available, only, in, bugis, n, great, world, la, e, buffet, cine, there, got, amore, wat]
ok lar joking wif u oni	[ok, lar, joking, wif, u, oni]
free entry in 2 a wkly comp to win fa cup final tkts 21st may 2005 text fa to 87121 to receive entry questionstd txt ratetcs apply 08452810075over18s	[free, entry, in, 2, a, wkly, comp, to, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, to, 87121, to, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]
u dun say so early hor u c already then say	[u, dun, say, so, early, hor, u, c, already, then, say]
nah i dont think he goes to usf he lives around here though	[nah, i, dont, think, he, goes, to, usf, he, lives, around, here, though]

Tokenization — Types & Python Code

Topic 5

Input: "NLP makes machines understand human language."

Word Tokenization:

NLP

makes

machines

understand

Types of Tokenization:

Word	["NLP","makes",...]	Classic NLP tasks
Sentence	["Hi.", "How are you?"]	Summarization
Subword	"playing"→["play", "##ing"]	BERT, GPT, T5
Character	"NLP"→["N", "L", "P"]	OCR, morphology

```
python — NLTK tokenizers

from nltk.tokenize import
    word_tokenize, sent_tokenize,
    RegexpTokenizer

text = "I love NLP. It's amazing!"

# Word tokens
word_tokenize(text)
# ['I', 'love', 'NLP', '.', 'It', 's', 'amazing', '!']

# Sentence tokens
sent_tokenize(text)
# ['I love NLP.', "It's amazing!"]

# Custom pattern (only words)
tok = RegexpTokenizer(r'\w+')
tok.tokenize(text)
# ['I', 'love', 'NLP', 'It', 's', 'amazing']
```

Methods to Perform Tokenization in Python

- Tokenization using Python's `split()` function
- Tokenization using Regular Expressions (Regex)
- Tokenization using NLTK, spaCy library, Keras, Gensim

Tokenization using Split()

- **split()** method as it is the most basic one. It returns a list of strings after breaking the given string by the specified separator. By default, split() breaks a string at each space. We can change the separator to anything.

Word Tokenization

```
1 text = ""Founded in 2002, SpaceX's mission is to enable humans to become a spacefaring civilization and a multi-p
2 species by building a self-sustaining city on Mars. In 2008, SpaceX's Falcon 1 became the first privately develop
3 liquid-fuel launch vehicle to orbit the Earth.""
4 # Splits at space
5 text.split()
```

split1.py hosted with ❤ by GitHub

[view raw](#)

Output : ['Founded', 'in', '2002,', 'SpaceX's', 'mission', 'is', 'to', 'enable', 'humans', 'to', 'become', 'a', 'spacefaring', 'civilization', 'and', 'a', 'multi-planet', 'species', 'by', 'building', 'a', 'self-sustaining', 'city', 'on', 'Mars.', 'In', '2008,', 'SpaceX's', 'Falcon', '1', 'became', 'the', 'first', 'privately', 'developed', 'liquid-fuel', 'launch', 'vehicle', 'to', 'orbit', 'the', 'Earth.']]

Sentence Tokenization

This is similar to word tokenization. Here, we study the structure of sentences in the analysis. A sentence usually ends with a full stop (.), so we can use "." as a separator to break the string:

```
1 text = ""Founded in 2002, SpaceX's mission is to enable humans to become a spacefaring civilization and a multi-p
2 species by building a self-sustaining city on Mars. In 2008, SpaceX's Falcon 1 became the first privately develop
3 liquid-fuel launch vehicle to orbit the Earth.""
4 # Splits at '.'
5 text.split('. ')
```

split2.py hosted with ❤ by GitHub

[view raw](#)

Output : ['Founded in 2002, SpaceX's mission is to enable humans to become a spacefaring civilization and a multi-planet \nspecies by building a self-sustaining city on Mars',
'In 2008, SpaceX's Falcon 1 became the first privately developed \nliquid-fuel launch vehicle to orbit the Earth.']

Tokenization using Regex

Regular expression is a sequence of character(s) mainly used to find and replace patterns in a string or file

The most common uses of regular expressions are:

- Search a string (search and match)
- Finding a string (findall)
- Break string into a sub strings (split)
- Replace part of a string (sub)

Commonly used methods regex methods are under “re” library:

- 1.re.match()
- 2.re.search()
- 3.re.findall()
- 4.re.split()
- 5.re.sub()
- 6.re.compile()

Return all words of a string those starts with vowel

Solution-1 Return each words

Code

```
result=re.findall(r'\w+', 'AV is largest Analytics community of India')  
print result
```

Output:

```
['AV', 'is', 'largest', 'Analytics', 'community', 'of', 'India']
```

Split a string with multiple delimiters

Solution

Code

```
import re  
line = 'asdf fjdk;afed,fjek,asdf,foo' # String has multiple delimiters (";",","," ","").  
result= re.split(r'[;,\s]', line)  
print result
```

Output:

```
['asdf', 'fjdk', 'afed', 'fjek', 'asdf', 'foo']
```

RegEx Functions

Function	Description
<u>findall</u>	Returns a list containing all matches
<u>search</u>	Returns a <u>Match object</u> if there is a match anywhere in the string
<u>split</u>	Returns a list where the string has been split at each match
<u>sub</u>	Replaces one or many matches with a string

Metacharacters

Character	Description	Example
[]	A set of characters	"[a-m]"
\	Signals a special sequence (can also be used to escape special characters)	"\d"
.	Any character (except newline character)	"he..o"
^	Starts with	"^hello"
\$	Ends with	"planet\$"
*	Zero or more occurrences	"he.*o"
+	One or more occurrences	"he.+o"
?	Zero or one occurrences	"he.?o"
{}	Exactly the specified number of occurrences	"he.{2}o"
	Either or	"falls stays"
()	Capture and group	

Special Sequences

Character	Description	Example
\A	Returns a match if the specified characters are at the beginning of the string	"\AThe"
\b	Returns a match where the specified characters are at the beginning or at the end of a word (the "r" in the beginning is making sure that the string is being treated as a "raw string")	r"\bain" r"ain\b"
\B	Returns a match where the specified characters are present, but NOT at the beginning (or at the end) of a word (the "r" in the beginning is making sure that the string is being treated as a "raw string")	r"\Bain" r"ain\B"
\d	Returns a match where the string contains digits (numbers from 0-9)	"\d"
\D	Returns a match where the string DOES NOT contain digits	"\D"
\s	Returns a match where the string contains a white space character	"\s"
\S	Returns a match where the string DOES NOT contain a white space character	"\S"
\w	Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore _ character)	"\w"
\W	Returns a match where the string DOES NOT contain any word characters	"\W"
\Z	Returns a match if the specified characters are at the end of the string	"Spain\Z"

Sets

Set	Description
[arn]	Returns a match where one of the specified characters (a , r , or n) is present
[a-n]	Returns a match for any lower case character, alphabetically between a and n
[^arn]	Returns a match for any character EXCEPT a , r , and n
[0123]	Returns a match where any of the specified digits (0 , 1 , 2 , or 3) are present
[0-9]	Returns a match for any digit between 0 and 9
[0-5][0-9]	Returns a match for any two-digit numbers from 00 and 59
[a-zA-Z]	Returns a match for any character alphabetically between a and z , lower case OR upper case
[+]	In sets, + , * , . , , () , \$, {} has no special meaning, so [+] means: return a match for any + character in the string

Steps and tools text preprocessing

Cleaning

Lowercasing

Removing Punctuation

Removing Hashtags and digits

Removing stop words

Tokenization

White Space Tokenizer

Word tokenizer

Sentence Tokenizer

Word punctuation Tokenizer

Tree bank word Tokenizer

Tweet Tokenizer

Natural Language Toolkit (NLTK)

Text Blob Word Tokenize

Text Blob Python library

spaCy Tokenizer

SpaCy open-source Python library

Gensim word tokenizer

Gensim Python library

Tokenization with Keras

Keras open-source library

Stemming

Porter Stemmer

Snowball Stemmer

Krovetz Stemmer

Lovin's Stemmer

Xerox Stemmer

Dawson's Stemmer

N-Gram Stemmer

Lancaster Stemmer

Lemmatization

WordNetLemmatizer (NLTK)

spaCy Lemmatization

Text Blob Lemmatizer

TreeTagger

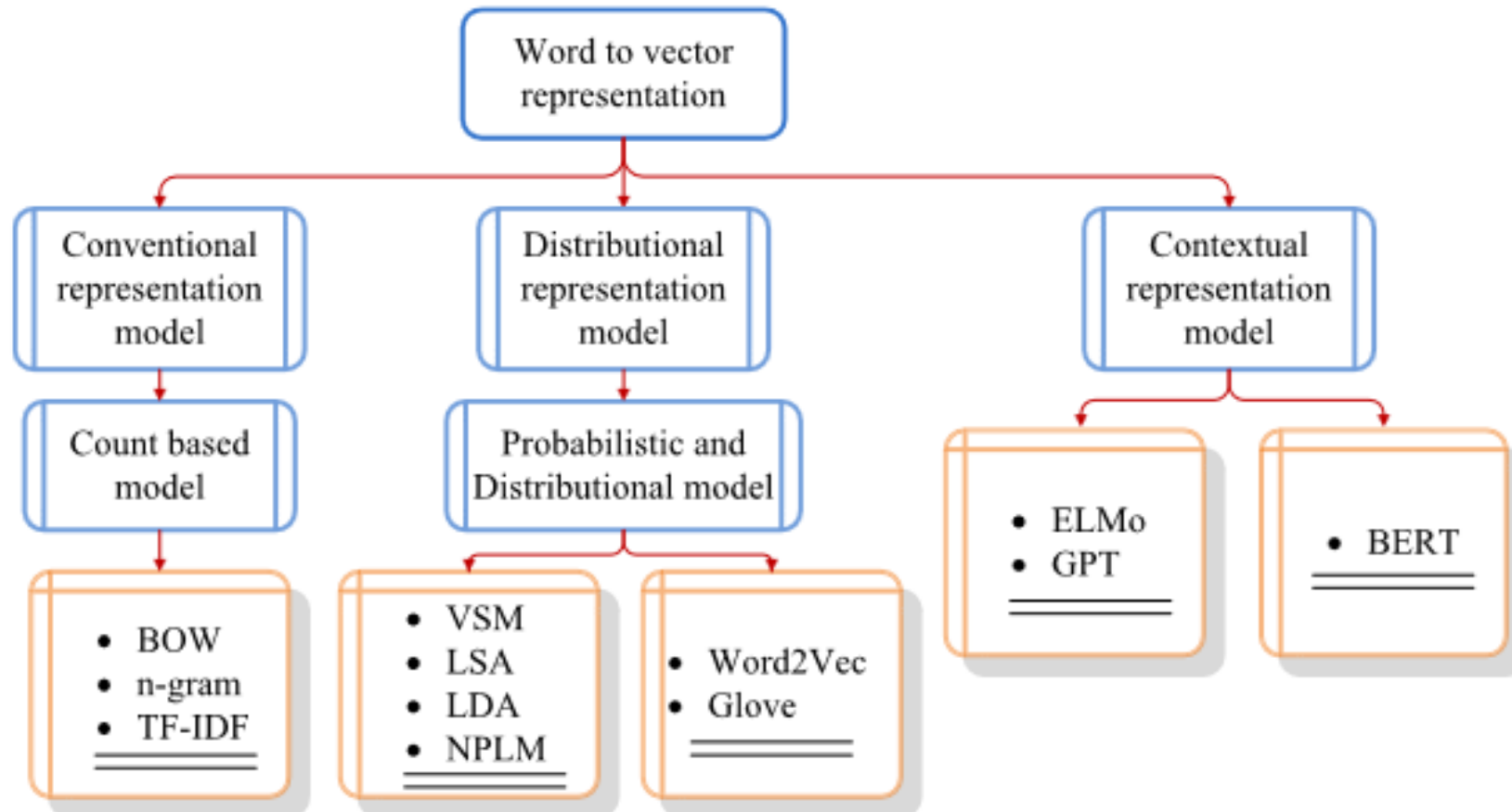
Pattern Lemmatizer (CLiPs)

Stanford CoreNLP Lemmatization

Gensim Lemmatize

Vectorization

- Word Embeddings or Word vectorization is a methodology in NLP to map words or phrases from vocabulary to a corresponding vector of real numbers which used to find word predictions, word similarities/semantics.



Word Embeddings — Words as Vectors + Code

Topic 5

Famous Analogy: king – man + woman ≈ queen (Word2Vec, Mikolov et al. 2013)

One-Hot

Sparse, no similarity
[0,0,1,0,0,...,0]

Word2Vec

Dense 300-d, semantic
[0.12,-0.45,0.89...]

BERT

Contextual: same word
different vectors

 python — Gensim Word2Vec embeddings

```
from gensim.models import Word2Vec

sentences = [
    ["king", "rules", "the", "land"],
    ["queen", "rules", "the", "kingdom"]
]

model = Word2Vec(sentences,
                  vector_size=100,
                  window=5, min_count=1)

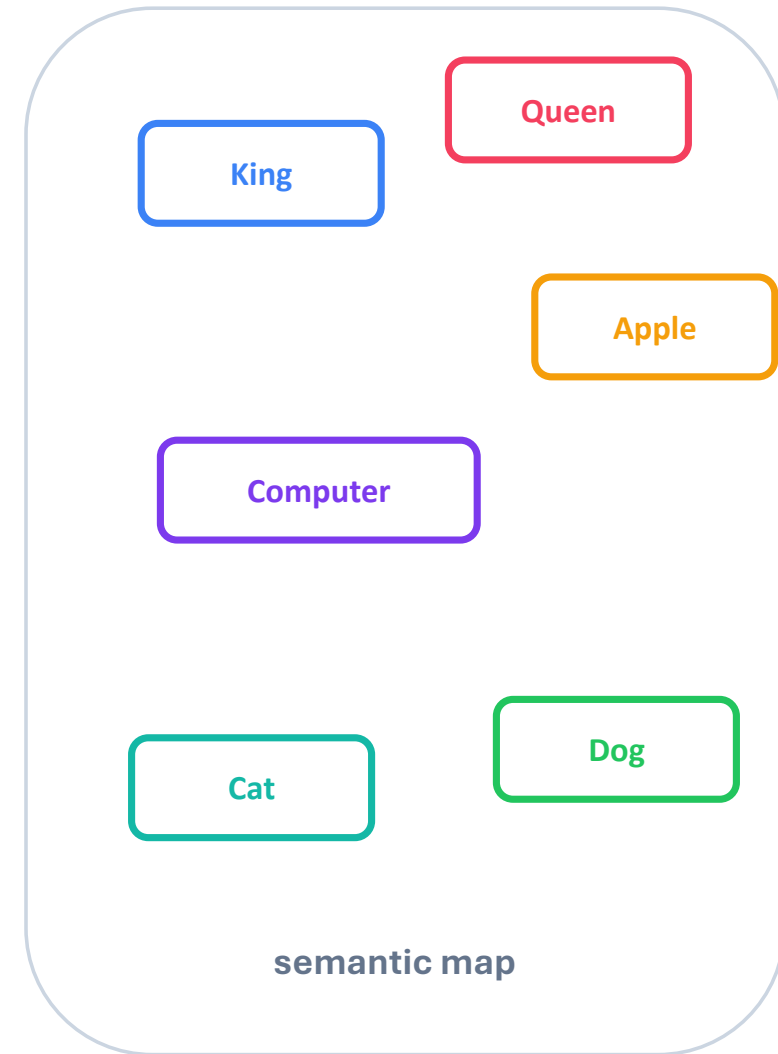
# Get vector for 'king'
model.wv['king'] # shape: (100,)

# Most similar words
model.wv.most_similar('king', topn=3)
# [('queen', 0.97), ('land', 0.82)...]
```

Word Embeddings

Words as Vectors

How computers move from text labels to meaning-rich numerical representations.



What Is Word Embedding?

A word embedding is a numerical vector that represents a word's meaning.

Earlier, computers treated words simply as text.

Examples: King, Queen, Apple, Computer.

A computer does not understand their meanings directly.

So we convert each word into numbers called vectors.

These numerical representations are called word embeddings.

Think of each word as coordinates in a high-dimensional space.

Words with similar meanings are located close together.



From Words to Coordinates

Embeddings let the computer compare meaning using vector distance.

Word	Vector Example
King	[0.25, 0.48, ...]
Man	[0.10, 0.32, ...]
Woman	[0.18, 0.44, ...]

A vector is just a list of numbers.

Each number captures part of how the word behaves in language.

The full vector becomes a position in meaning space.

Nearby positions usually mean related words.

Key Idea

The computer does not store dictionary definitions. It learns useful patterns from word usage.

Famous Word2Vec Analogy

King - Man + Woman is very close to Queen.

King - **Man** + **Woman** approximately **Queen**

This demonstrates that Word2Vec learned relationships between words.

It did not merely memorize word labels.

The vector direction from Man to Woman is similar to the direction from King to Queen.

One-Hot Encoding

A simple representation where each word gets its own position.

Word	Vector
Cat	[1 0 0 0]
Dog	[0 1 0 0]
Car	[0 0 1 0]
Apple	[0 0 0 1]

Example vocabulary

Cat
Dog
Car
Apple

Identity is captured, but meaning is not.

Problem With One-Hot Encoding

Every word is equally different.

$$\text{Distance}(\text{Cat}, \text{Dog}) = \text{Distance}(\text{Cat}, \text{Apple})$$

The computer cannot understand that Cat and Dog are both animals.
Cat and Apple look just as different as Cat and Dog.
Therefore, one-hot encoding captures identity only, not meaning.

Word2Vec

Instead of huge sparse vectors, each word gets a dense vector.

King

```
[ 0.23  
-0.61  
0.89  
0.42  
... ]
```

Dense vectors are compact and meaning-rich.

Nearby word groups

King, Queen, Prince, Princess are very close.

Doctor, Nurse, Hospital, Medicine are nearby.

The model learns this by observing surrounding words.

How Word2Vec Learns

It observes surrounding words again and again.

Context window



Words that often appear in similar contexts receive similar vectors.

If dog often appears near barks, puppy, plays, and pet, those words become related in vector space.

If hospital often appears near doctor, nurse, patient, and medicine, those vectors move closer.

BERT Embeddings

Modern embeddings are context aware.

Sentence 1

He deposited money in the bank.

bank = financial institution

Sentence 2

The fisherman sat on the river bank.

bank = side of a river

Word2Vec gives one vector for bank. BERT gives different vectors based on context.

One-Hot vs Word2Vec vs BERT

The main evolution is from identity to meaning to context.

Method	Meaning Captured	Context Aware
One-Hot	No	No
Word2Vec	Yes	No
BERT	Yes	Yes

BERT understands language much better because word meaning can change from sentence to sentence.

More Famous Word2Vec Examples

Students remember embeddings when relationships become visible.

Paris - France + Italy

approximately

Rome

Tokyo - Japan + India

approximately

Delhi

Walking - Walk + Swim

approximately

Swimming

Brother - Man + Woman

approximately

Sister

These examples demonstrate that embeddings encode semantic relationships.

Code Example 1: Find Similar Words

Train a small Word2Vec model and ask for words similar to dog.

```
from gensim.models import Word2Vec

sentences = [
    ["dog", "barks", "loudly"],
    ["cat", "meows"],
    ["puppy", "plays"],
    ["dog", "plays"],
    ["cat", "sleeps"],
    ["puppy", "barks"]
]

model = Word2Vec(
    sentences,
    vector_size=50,
    window=2,
    min_count=1
)

print(model.wv.most_similar("dog"))
```

Output may look like

```
[
    ('puppy', 0.91),
    ('barks', 0.74),
    ('plays', 0.61)
]
```

The model sees dog, puppy, barks, and plays in related contexts.

Code Example 2: Word Analogy

Use vector arithmetic to solve an analogy.

```
from gensim.models import KeyedVectors

# Example with trained or pretrained vectors
result = model.wv.most_similar(
    positive=['queen', 'man'],
    negative=['woman'],
    topn=5
)

print(result)

# Common pretrained-vector analogy
result = model.wv.most_similar(
    positive=['king', 'woman'],
    negative=['man']
)

print(result)
```

Expected output

queen

The exact result depends on the training data and vector quality.

Code Example 3: Cosine Similarity

Semantically similar words have higher cosine similarity.

```
print(model.wv.similarity("dog", "puppy"))  
print(model.wv.similarity("dog", "car"))
```

Example output

Dog-Puppy : 0.89
Dog-Car : 0.14

Cosine similarity compares the angle between vectors. Similar meanings point in similar directions.

Classroom Explanation: 2 Minutes

A ready-to-speak explanation for students.

"Imagine every word as a point on a map. In One-Hot Encoding, every word is isolated, so the computer knows only that they are different. Word2Vec changes this by placing similar words close together - 'king' near 'queen', 'doctor' near 'hospital', and 'cat' near 'dog'. BERT goes a step further: it understands that the same word can have different meanings depending on the sentence. For example, 'bank' in 'river bank' and 'bank' in 'bank account' are represented differently. This evolution - from One-Hot to Word2Vec to BERT - is one of the biggest milestones in modern NLP."

This connects the concept with intuitive examples and practical code students can relate to.

Key Takeaways

The full story in one slide.

One-Hot

Huge sparse vector
Captures identity
Does not capture meaning

Word2Vec

Dense vector
Captures meaning
One vector per word

BERT

Dense contextual vector
Captures meaning
Understands context

Word embeddings changed NLP by turning words into mathematical objects that can be compared, clustered, and reasoned with.

Stemming & POS Tagging

Reducing words to roots and labeling grammatical roles

06

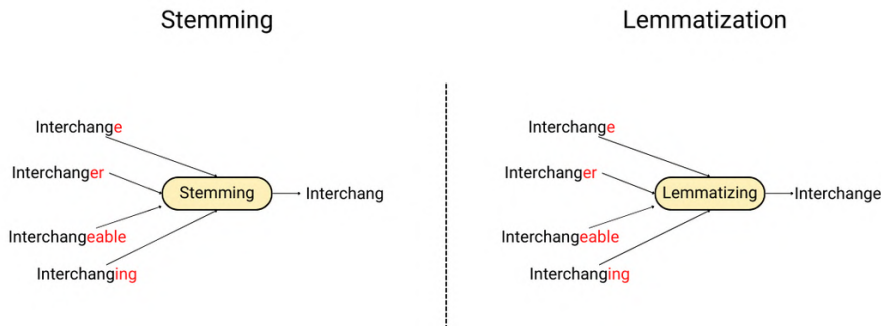
Stemming

- It is also known as the text standardization step where the words are stemmed or diminished to their root/base form. **For example,** words like 'programmer', 'programming', 'program' will be stemmed to 'program'.
- But the **disadvantage** of stemming is that it stems the words such that its root form loses the meaning or it is not diminished to a proper English word.

no_stopwords	msg_stemmed
[go, jurong, point, crazy, available, bugis, n, great, world, la, e, buffet, cine, got, amore, wat]	[go, jurong, point, crazi, avail, bugi, n, great, world, la, e, buffet, cine, got, amor, wat]
[ok, lar, joking, wif, u, oni]	[ok, lar, joke, wif, u, oni]
[free, entry, 2, wkly, comp, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, 87121, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]	[free, entri, 2, wkli, comp, win, fa, cup, final, tkt, 21st, may, 2005, text, fa, 87121, receiv, entri, questionstd, txt, ratetc, appli, 08452810075over18]
[u, dun, say, early, hor, u, c, already, say]	[u, dun, say, earli, hor, u, c, alreadi, say]
[nah, dont, think, goes, usf, lives, around, though]	[nah, dont, think, goe, usf, live, around, though]

Lemmatization

- It stems the word but makes sure that it does not lose its meaning. Lemmatization has a pre-defined dictionary that stores the context of words and checks the word in the dictionary while diminishing.



no_stopwords	msg_stemmed	msg_lemmatized
[go, jurong, point, crazy, available, bugis, n, great, world, la, e, buffet, cine, got, amore, wat]	[go, jurong, point, crazi, avail, bugi, n, great, world, la, e, buffet, cine, got, amor, wat]	[go, jurong, point, crazy, available, bugis, n, great, world, la, e, buffet, cine, got, amore, wat]
[ok, lar, joking, wif, u, oni]	[ok, lar, joke, wif, u, oni]	[ok, lar, joking, wif, u, oni]
[free, entry, 2, wkly, comp, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, 87121, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]	[free, entri, 2, wkli, comp, win, fa, cup, final, tkt, 21st, may, 2005, text, fa, 87121, receiv, entri, questionstd, txt, ratetc, appli, 08452810075over18]	[free, entry, 2, wkly, comp, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, 87121, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]
[u, dun, say, early, hor, u, c, already, say]	[u, dun, say, earli, hor, u, c, already, say]	[u, dun, say, early, hor, u, c, already, say]
[nah, dont, think, goes, usf, lives, around, though]	[nah, dont, think, goe, usf, live, around, though]	[nah, dont, think, go, usf, life, around, though]

Stemming vs Lemmatization — Side by Side

Topic 6

Stemming (Porter / Snowball)

Rule-based — fast but may produce non-words:

running → *run*

studies → *studi*

flies → *fli*

better → *better*

Lemmatization (WordNet)

Vocabulary-based — slower but always valid words:

running → *run*

better → *good*

studies → *study*

corpora → *corpus*

python — Stemming & Lemmatization (NLTK)

```
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer

stemmer = PorterStemmer()
lemma = WordNetLemmatizer()

words = ["running", "studies", "better", "corpora"]

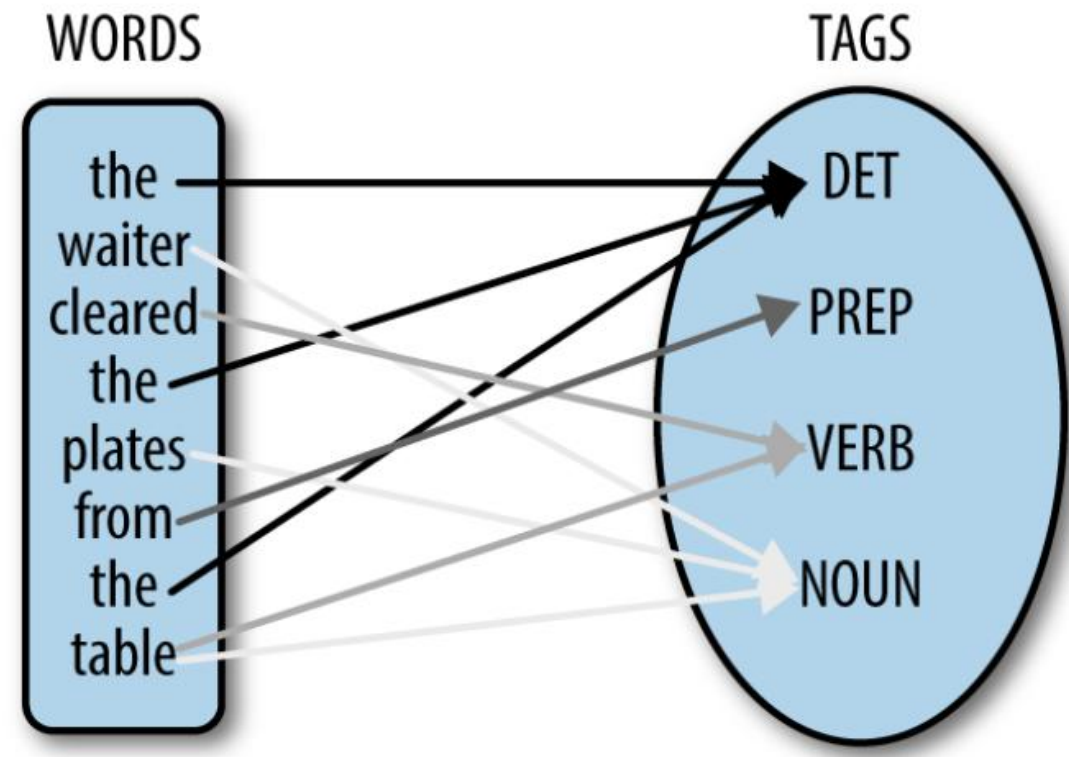
for w in words:
    s = stemmer.stem(w)
    l = lemma.lemmatize(w, pos='v')
    print(f"{w:<10} stem:{s:<10} lem:{l}")

# running    stem:run      lem:run
# studies    stem:studi   lem:study
# better     stem:better  lem:good
# corpora    stem:corpora lem:corpus
```

POS Tagging

- Part-of-speech-tagging is a labeling/tagging mechanism where the words in the given corpus is tagged grammatically, it means the words are labeled as a noun, verb, adjective, adverb, etc. It even does fine-grained tagging like 'noun-plural'. It even considers tense while tagging.

- POS Tagging Application:**
 - Text 2 Speech recognition
 - Word sense disambiguation



Parts of Speech in English

There are several **POS Tagsets**

Most modern language processing on English uses the **45-tag Penn Treebank** tagset (Marcus et al., 1993) as show in the table

Other tagsets:

Brown corpus: 87 POS tags

C5 tagset: 61 POS tags

Tag	Description	Example	Tag	Description	Example
CC	coordin. conjunction	<i>and, but, or</i>	SYM	symbol	<i>+, %, &</i>
CD	cardinal number	<i>one, two</i>	TO	“to”	<i>to</i>
DT	determiner	<i>a, the</i>	UH	interjection	<i>ah, oops</i>
EX	existential ‘there’	<i>there</i>	VB	verb base form	<i>eat</i>
FW	foreign word	<i>mea culpa</i>	VBD	verb past tense	<i>ate</i>
IN	preposition/sub-conj	<i>of, in, by</i>	VBG	verb gerund	<i>eating</i>
JJ	adjective	<i>yellow</i>	VCN	verb past participle	<i>eaten</i>
JJR	adj., comparative	<i>bigger</i>	VBP	verb non-3sg pres	<i>eat</i>
JJS	adj., superlative	<i>wildest</i>	VBZ	verb 3sg pres	<i>eats</i>
LS	list item marker	<i>I, 2, One</i>	WDT	wh-determiner	<i>which, that</i>
MD	modal	<i>can, should</i>	WP	wh-pronoun	<i>what, who</i>
NN	noun, sing. or mass	<i>llama</i>	WP\$	possessive wh-	<i>whose</i>
NNS	noun, plural	<i>llamas</i>	WRB	wh-adverb	<i>how, where</i>
NNP	proper noun, sing.	<i>IBM</i>	\$	dollar sign	<i>\$</i>
NNPS	proper noun, plural	<i>Carolinas</i>	#	pound sign	<i>#</i>
PDT	predeterminer	<i>all, both</i>	“	left quote	<i>‘ or “</i>
POS	possessive ending	<i>’s</i>	”	right quote	<i>’ or ”</i>
PRP	personal pronoun	<i>I, you, he</i>	(	left parenthesis	<i>[, (, {, <</i>
PRP\$	possessive pronoun	<i>your, one’s</i>	)	right parenthesis	<i>],), }, ></i>
RB	adverb	<i>quickly, never</i>	,	comma	<i>,</i>
RBR	adverb, comparative	<i>faster</i>	.	sentence-final punc	<i>. ! ?</i>
RBS	adverb, superlative	<i>fastest</i>	:	mid-sentence punc	<i>: ; ... - -</i>
RP	particle	<i>up, off</i>			

References:

Rule-based POS Tagging

Depends on a **dictionary** that provides possible **POS tags** for a word,
or **rules** can be **learned** using **training data**

Ambiguity can be **removed** using **manually** developed **rules**

Example Rule:

if preceding word is ART:

disambiguate {NOUN, VERB} as NOUN.

Statistical POS Tagging

Involves **selecting** most likely **sequence of tags** for words

We need to calculate $P(T1...Tn \mid w1...wn)$

Using **Bayes Rule** this is equal to:

$$\frac{P(T1...Tn \mid w1...wn) = P(T1...Tn) * P(w1...wn \mid T1...Tn)}{P(w1...wn)}$$

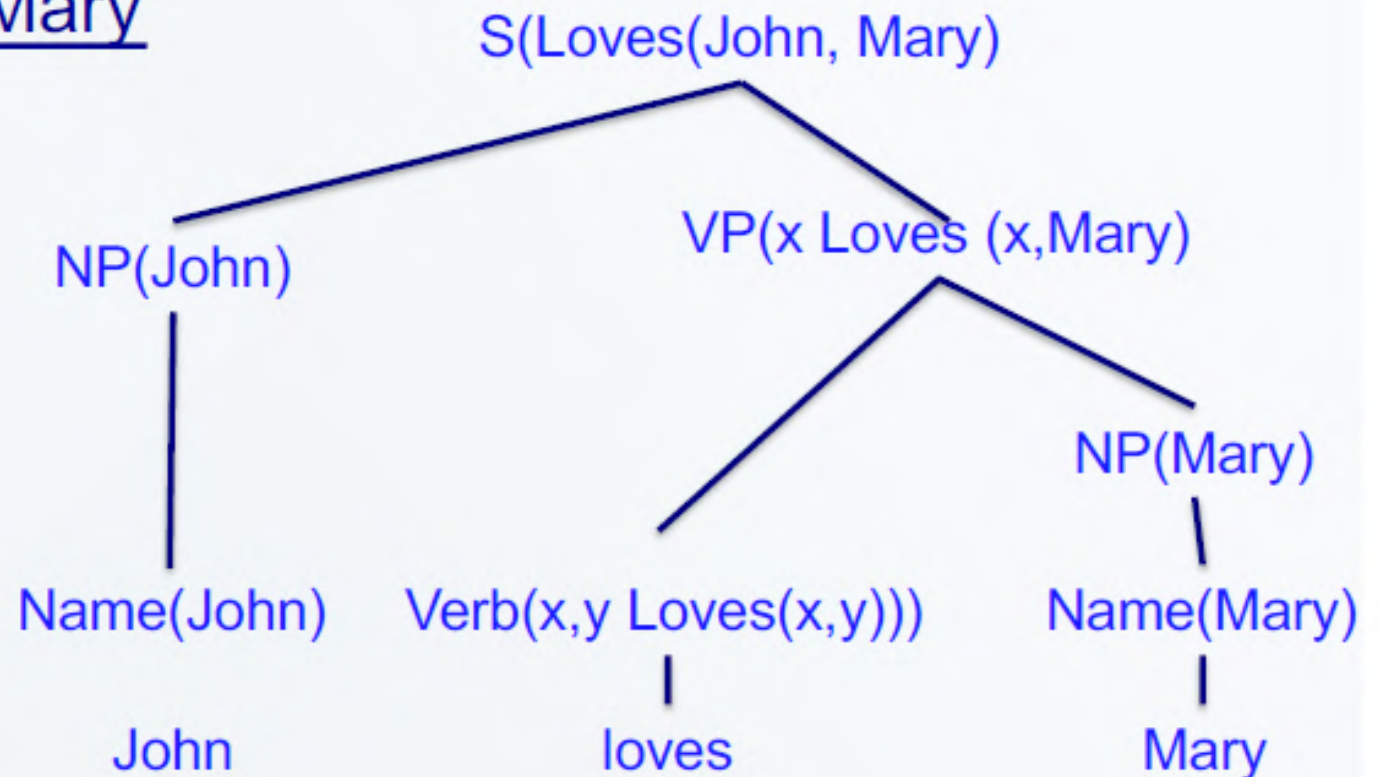
Calculating the above **probability** requires a lot of data

So, we approximate this by assuming **independence** based on part-of-speech **tag bigrams** and **lexical** generation probabilities

Assume input sentence **S** in natural language **L**. Assume you have rules (*grammar* **G**) that describe syntactic regularities (patterns or structures). Given **S** & **G**, find syntactic structure of **S**. Such a structure is called a **Parse Tree**

Pars tree: John loves Mary

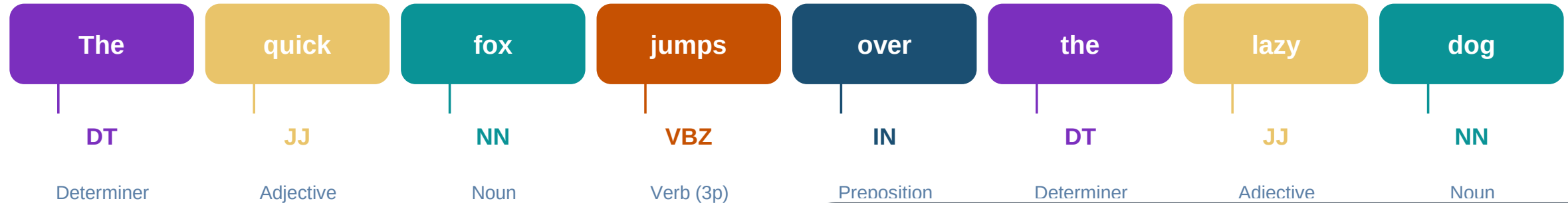
Helps a computer to automatically answer questions like -Who did what and when?



POS Tagging — Annotating Grammatical Roles

Topic 6

Example sentence:



Common Penn Treebank Tags:

- ▶ NN / NNS = Noun / Plural
- ▶ VB / VBZ = Verb base / 3rd person
- ▶ JJ = Adjective, RB = Adverb
- ▶ DT = Determiner, IN = Preposition
- ▶ PRP = Pronoun, CC = Conjunction

```
python — POS Tagging (NLTK & spaCy)

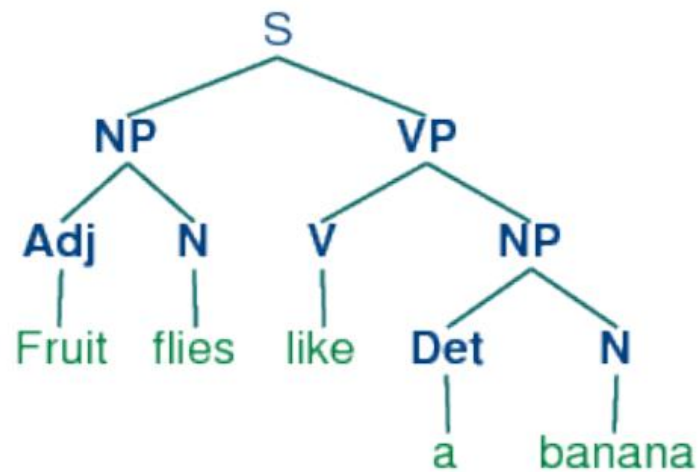
import nltk
from nltk.tokenize import word_tokenize

# NLTK POS Tagging
text = "The quick fox jumps"
tokens = word_tokenize(text)
tags = nltk.pos_tag(tokens)
# [('The', 'DT'), ('quick', 'JJ'), ...

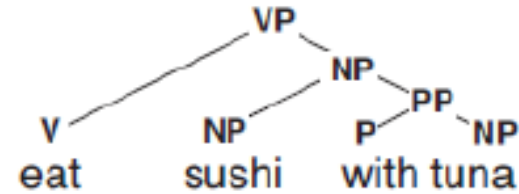
# spaCy POS Tagging (more accurate)
import spacy
nlp = spacy.load('en_core_web_sm')
doc = nlp("The quick fox jumps")
for tok in doc:
    print(tok.text, tok.pos_, tok.tag_)

# The DET DT
# quick ADJ JJ | fox NOUN NN
```

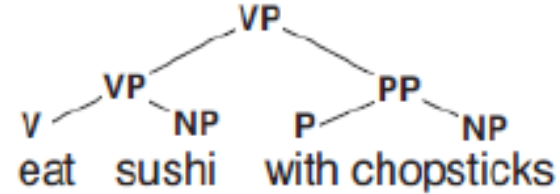
Syntactics Parsing



Correct analysis

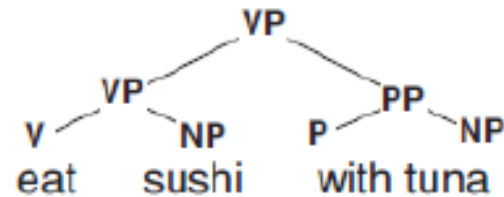


eat sushi with tuna

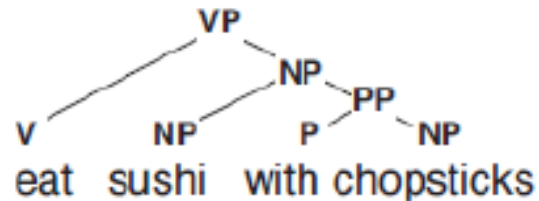


eat sushi with chopsticks

Incorrect analysis



eat sushi with tuna



eat sushi with chopsticks

Syntactic analysis or parsing

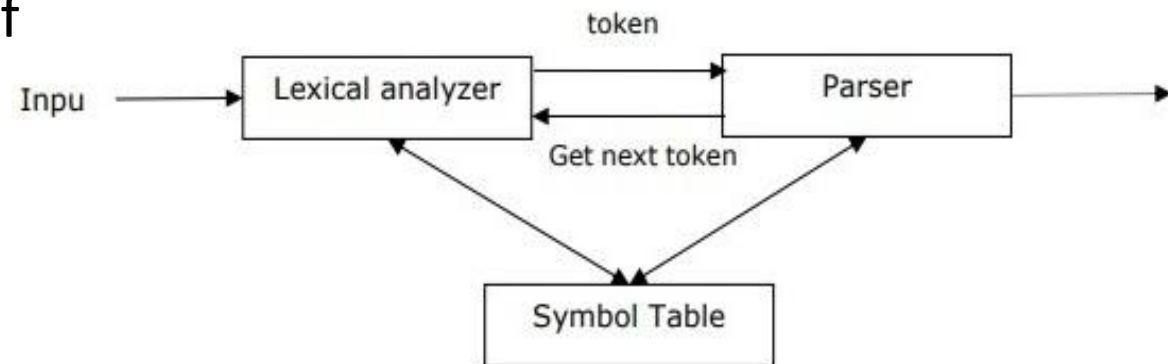
- The purpose of this phase is to draw exact meaning, or dictionary meaning from the text.
- Syntax analysis checks the text for meaningfulness comparing to the rules of formal grammar.
- For example, the sentence like “hot ice-cream” would be rejected by semantic analyzer.
- In this sense, syntactic analysis or parsing may be defined as the process of analyzing the strings of symbols in natural language conforming to the rules of formal grammar.

Concept of Parser

- It is defined as the software component designed for taking input data (text) and giving structural representation of the input after checking for correct syntax as per formal grammar.

The main roles of the parse include –

- To report any syntax error.
- To recover from commonly occurring error so that the processing of the remainder of program can be continued.
- To create parse tree.
- To create symbol table.
- To produce intermediate representations (IR).



Types of Parsing

- Derivation divides parsing into the followings two types –

Top-down Parsing

- In this kind of parsing, the parser starts constructing the parse tree from the start symbol and then tries to transform the start symbol to the input.
- The most common form of top-down parsing uses recursive procedure to process the input.
- The main disadvantage of recursive descent parsing is backtracking.

Bottom-up Parsing

In this kind of parsing, the parser starts with the input symbol and tries to construct the parser tree up to the start symbol.

Types of Derivation

- To decide which non-terminal to be replaced with production rule
- **Left-most Derivation**
- In the left-most derivation, the sentential form of an input is scanned and replaced from the left to the right. The sentential form in this case is called the left-sentential form.
- **Right-most Derivation**
- In the right-most derivation, the sentential form of an input is scanned and replaced from right to left. The sentential form in this case is called the right-sentential form.

Concept of Parse Tree

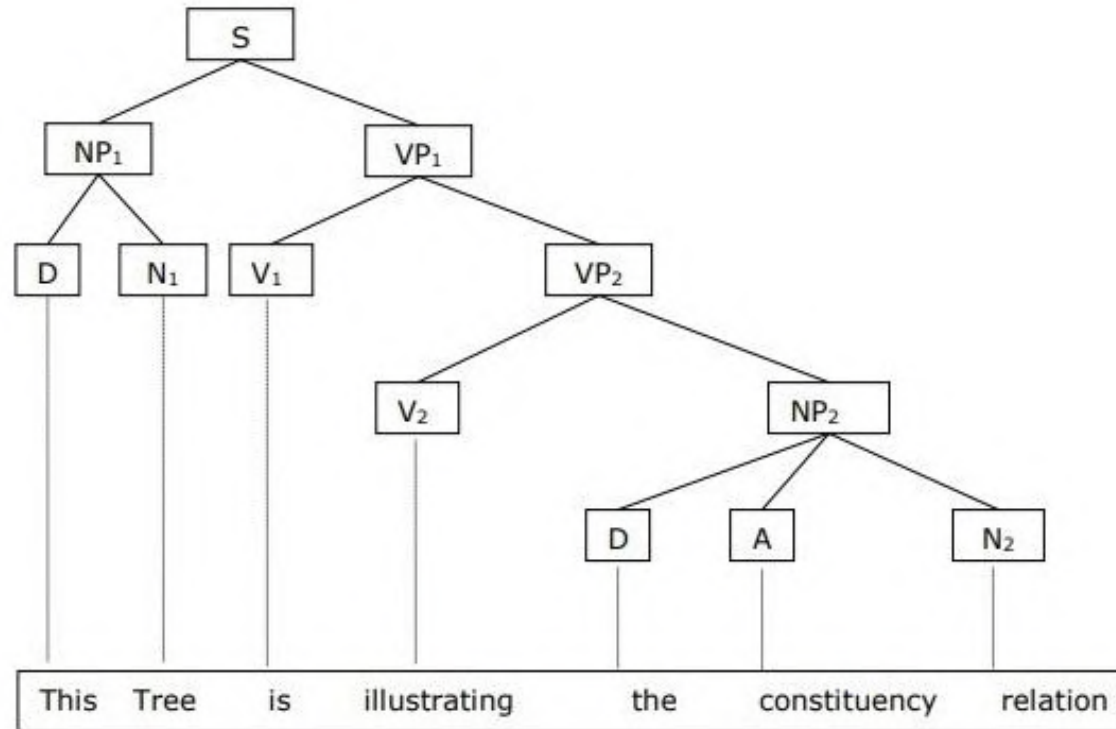
- It is the graphical depiction of a derivation.
- The start symbol of derivation serves as the root of the parse tree.
- In every parse tree, the leaf nodes are terminals and interior nodes are non-terminals.
- A property of parse tree is that in-order traversal will produce the original input string.

Concept of Grammar

- Grammar is very essential and important to describe the syntactic structure of well-formed programs.
- A mathematical model of grammar was given by **Noam Chomsky** in 1956, which is effective for writing computer languages.
- Mathematically, a grammar G can be formally written as a 4-tuple (N, T, S, P) where
 - N or V_N = set of non-terminal symbols, i.e., variables.
 - T or Σ = set of terminal symbols.
 - S = Start symbol where $S \in N$
 - P denotes the Production rules for Terminals as well as Non-terminals. It has the form $\alpha \rightarrow \beta$, where α and β are strings on $V_N \cup \Sigma$ and least one symbol of α belongs to V_N

Phrase Structure or Constituency Grammar

- Example : “This tree is illustrating the constituency relation”



Constituency Grammar Or Phrase Structure

```
1 import nltk
2 from nltk
3 import pos_tag, word_tokenize, RegexpParser
4
5 # String to parse
6 to_parse = "This tree is illustrating the constituency relation"
7
8 # Find all parts of speech in above sentence
9 tagged_parts = pos_tag(word_tokenize(to_parse))
10
11 # Defining grammar on basis of which we 've to extract
12 grammar = r ""
13 " NP: {<DT>?<JJ>*<NN>}
14 P: {
15     <IN>}
16     V: {<V.*>}
17     PP: {<p> <NP>}
18     VP: {<V> <NP|PP>*" ""
19
20 #Extracting all parts of speech
21 parser = RegexpParser(grammar)
22
23 # Print all parts of speech in above sentence
24 output = parser.parse(tagged_parts)
25 print("\nAfter Extracting the parts\n\n", output, "\n")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL JUPYTER

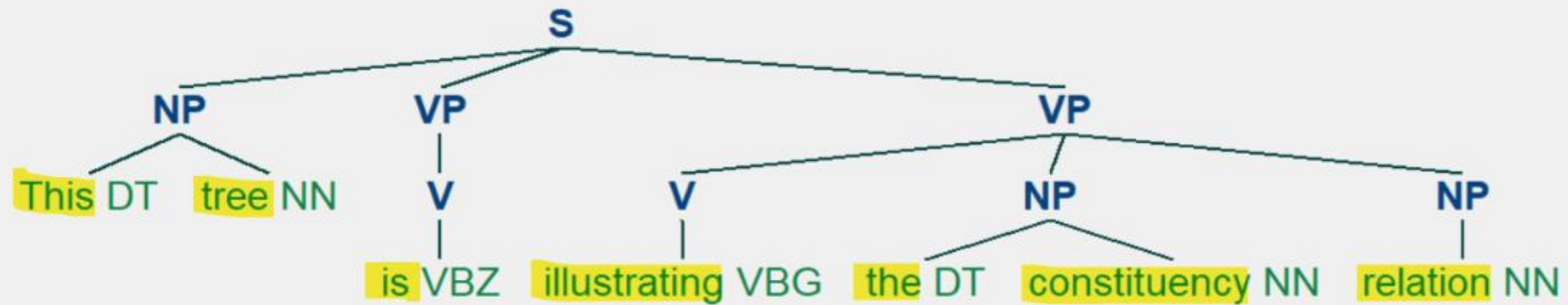
After Extracting the parts

```
(S
  (NP This/DT tree/NN)
  (VP (V is/VBZ))
  (VP
    (V illustrating/VBG)
    (NP the/DT constituency/NN)
    (NP relation/NN)))
```

PS D:\Druti> █

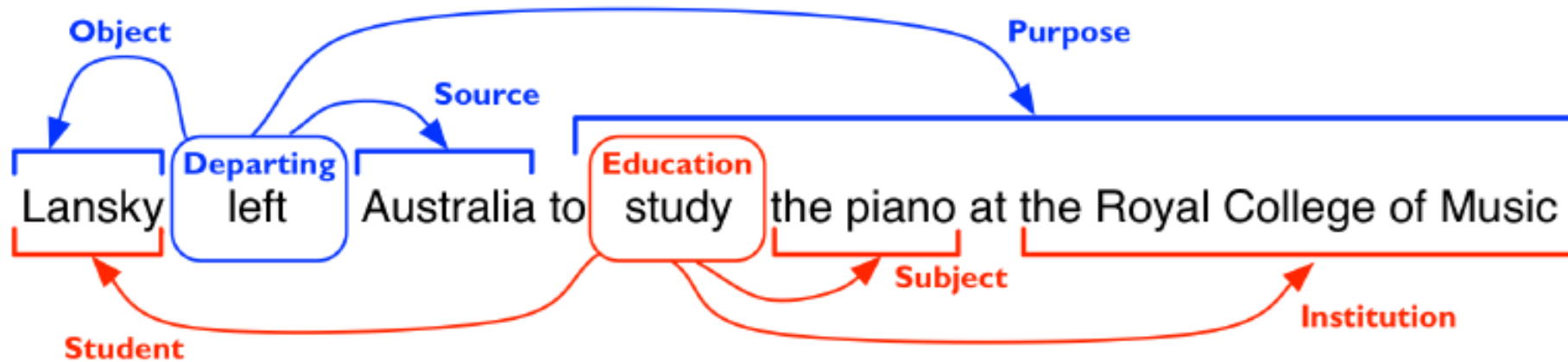
Graphical Representation Of The Above Output Of Code

- To get the graphical representation we have to run one more line of code i.e. **output.draw()**



Semantic Parsing

- ❖ Word sense disambiguation
- ❖ Semantic role labeling



Credit: Ivan Titov

UNIT 1

Named Entity Recognition (NER)

Finding and classifying real-world entities in text

07

Information Retrieval

- Information Retrieval (IR) is finding material (usually documents) of an unstructured nature (usually text) that satisfies an information need from within large collections (usually stored on computers).
- These days we frequently think first of web search, but there are many other cases:
 - E-mail search
 - Searching your laptop
 - Corporate knowledge bases
 - Legal information retrieval



Chunking

- **Chunking** involves grouping together individual pieces of information from a sentence, such as nouns, verbs, adjectives, and adverbs, into larger units known as chunks.
- The most common type of chunking is noun phrase (NP) chunking, which involves identifying and extracting noun phrases from a sentence, such as "the cat," "a book," or "my friend." Another type of chunking is verb phrase (VP) chunking, which involves identifying and extracting verb phrases from a sentence, such as "ate breakfast," "is running," or "will sing."
- **Steps:**
 - Identify the parts of speech in a sentence using part-of-speech (POS) tagging.
 - The chunks are created by grouping together certain types of words based on their POS tags.
- **The most common types of chunks include**
 - noun phrases
 - verb phrases,
 - prepositional phrases.

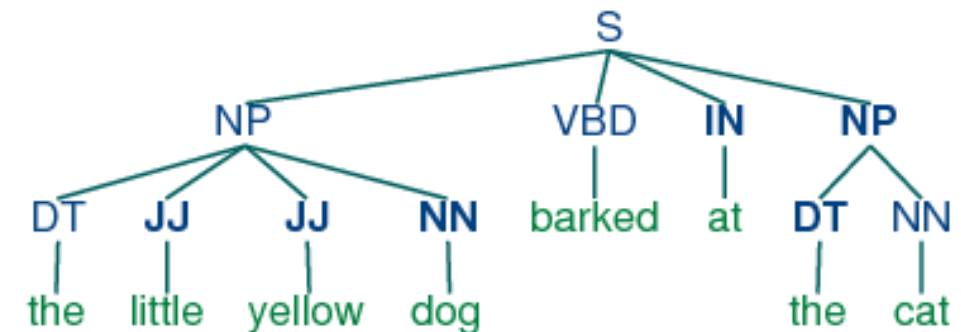
Here is an example of chunking in action:

Sentence: The cat sat on the mat.

POS tags: The/DT cat/NN sat/VBD on/IN the/DT mat/NN.

Chunks:

- Noun Phrase: The cat
- Verb Phrase: sat
- Prepositional Phrase: on the mat



Chinking

- **Chinking**, on the other hand, is the process of excluding certain words or phrases from a chunk. This is useful when we want to exclude specific types of words, such as prepositions, conjunctions, or determiners, from the chunks we extract.
- Chinking is typically used in combination with chunking, and it involves identifying the words or phrases that we want to exclude from a chunk and marking them with the tag O (outside) in a named entity recognition (NER) system.

Here is an example of chinking in action:

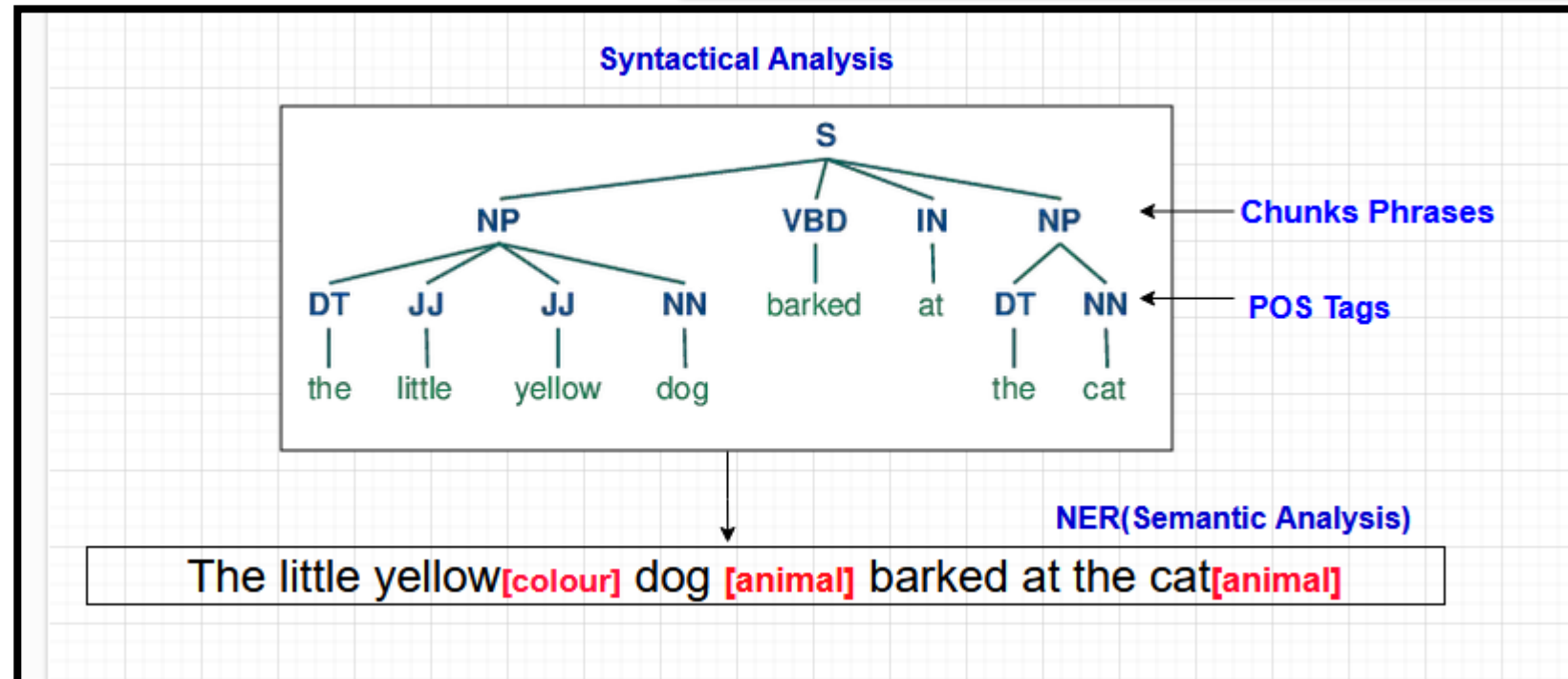
Sentence: The cat sat on the mat.

Chunk: The cat sat on the mat

Chink: Remove the prepositional phrase "on the mat"

Final Chunk: The cat sat

POS Tagging vs Chunking vs NER



Entities and Relations

- The essence of a document can often be captured by the entities and relations that are mentioned
- **Entity**: object, person, organization, date, etc.
 - Most things denoted by a noun phrase or pronoun
- **Relation**: property that links one or several entities
 - Most things denoted by an adjective, verb or adverb

Named Entity Recognition (NER)

- Task:
 - Identify named entities
 - Classify named entities
- Classes:
 - Common: **Person, organization, location, quantity, time, money, percentage, etc.**
 - Biology: genes, proteins, molecules, etc.

- **Identify** token spans of entity mentions in text, and **classify** them into types of interest

[Barack Obama] arrived this afternoon in [Washington, D.C].
[President Obama]'s wife [Michelle] accompanied him

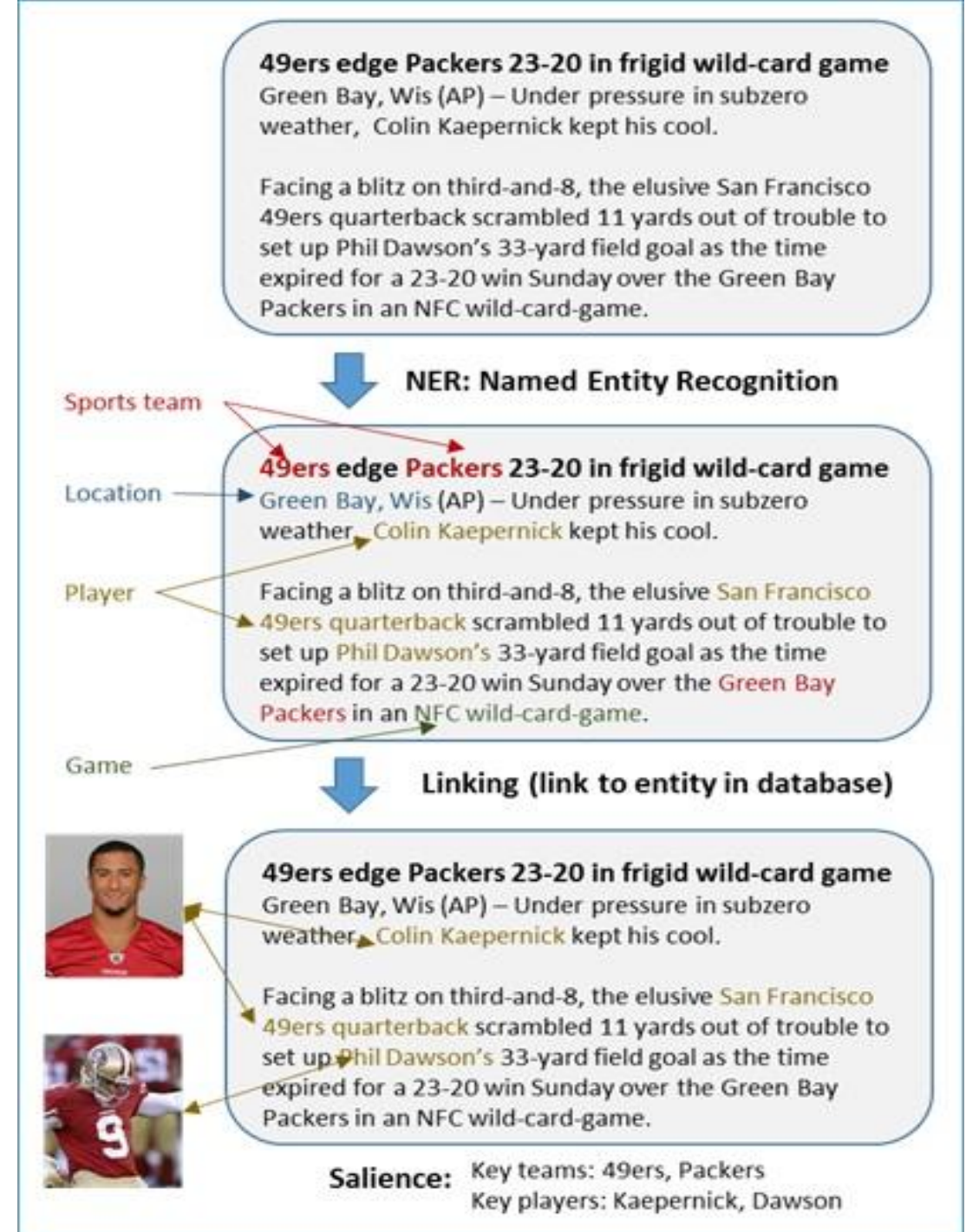
PERSON
LOCATION

[TNF alpha] is produced chiefly by activated [macrophages]

PROTEIN
CELL

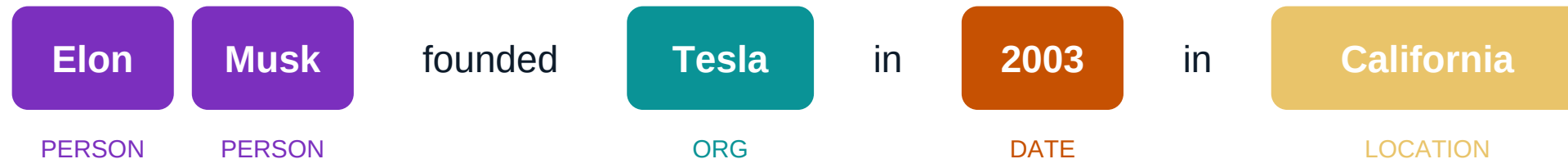
NER

- NER is the process of identifying and categorizing words, expressions, or names in unstructured data into predefined categories such as persons, organizations, and locations.
- **It is extremely helpful in:**
 - Identifying the names in a given social media content
 - Read between news article to extract the named entities like brand name, company name, etc
 - Simplifies search and makes it efficient by giving us the set of entities to be searched in the pool of corpus
 - We can easily pick the relevant location info from the given set of corpus to streamline our actions accordingly
 - It is used extensively in chatbots to plan the responses to be given to the user.



NER locates and classifies named entities into categories: **PERSON**, **ORG**, **LOCATION**, **DATE**, **MONEY**, **MISC**.

Example:



PERSON	Narendra Modi, Einstein
ORG	ISRO, Google, SIT
LOCATION	Pune, Amazon River, Mars
DATE	26 Jan, last Monday
MONEY	₹500 crore, \$1B

python — spaCy NER + displacy viz

```
import spacy
from spacy import displacy

nlp = spacy.load('en_core_web_sm')
doc = nlp("Elon Musk founded Tesla in 2003")

for ent in doc.ents:
    print(ent.text, ent.start_char,
          ent.end_char, ent.label_)

# Elon Musk 0 9 PERSON
```

NER Applications

- NER Categorizes Documents
- NER Enhances Customer Support
- NER Assists Biomedical Researchers and Hospitals
- NER Influences Recommendation Systems
- Content Classification for Media Agencies with NER
- Summarizes Resumes
- NER Provides Support to Chatbots
- NER to Optimize Search Engine Algorithms

Legal NLP

- Extracting DOC (Document Type), PARTY (entity signing a contract), ALIAS (the way a company is named later on in the document) and EFFDATE (Effective Date of the contract).

The screenshot displays a web-based interface for Legal NLP. On the left, a sidebar contains a dropdown menu labeled 'Choose the pretrained model to test' with 'legalner_parties' selected. Below this is a section titled 'Entity labels' showing five buttons: 'DOC', 'ALIAS', 'EFFDATE', and 'PARTY', each with an 'x' icon to remove it. The main area on the right is titled 'Choose Sample Text' and shows a dropdown menu with 'Exhibit 2.7 FORM OF TRADEMARK LICENSE AGREEMENT...'. Below this, the text 'Text annotated with identified Named Entities' is displayed. The sample text is: 'Exhibit 2.7 FORM OF TRADEMARK LICENSE AGREEMENT THIS TRADEMARK LICENSE AGREEMENT (this "Agreement"), made and entered into as of the July 1, 2020 (the "Effective Date"), by and between ARCONIC INC., a corporation organized under the laws of Delaware ("Licensee") and ARCONIC ROLLED PRODUCTS CORP., a corporation organized under the laws of Delaware ("Licensor")'. The text is annotated with colored boxes and labels: 'TRADEMARK LICENSE AGREEMENT' is highlighted in dark red with a 'DOC' label below it; 'July 1, 2020' is highlighted in green with an 'EFFDATE' label below it; 'ARCONIC INC.' is highlighted in light blue with a 'PARTY' label below it; 'ARCONIC ROLLED PRODUCTS CORP.' is highlighted in light blue with a 'PARTY' label below it; 'Licensee' is highlighted in purple with an 'ALIAS' label below it; and 'Licensor' is highlighted in purple with an 'ALIAS' label below it.

Choose the pretrained model to test

legalner_parties

Entity labels

DOC X ALIAS X

EFFDATE X PARTY X

Choose Sample Text

Exhibit 2.7 FORM OF TRADEMARK LICENSE AGREEMENT...

Text annotated with identified Named Entities

Exhibit 2.7 FORM OF TRADEMARK LICENSE AGREEMENT THIS TRADEMARK LICENSE AGREEMENT (this "Agreement"), made and entered into as of the July 1, 2020 (the "Effective Date"), by and between ARCONIC INC., a corporation organized under the laws of Delaware ("Licensee") and ARCONIC ROLLED PRODUCTS CORP., a corporation organized under the laws of Delaware ("Licensor")

DOC

DOC

EFFDATE

PARTY

PARTY

ALIAS

ALIAS

- Detecting clinical entities in text.

- Choose the pretrained model to test

ner_clinical

Entity labels

PROBLEM x

TREATMENT x

Try it yourself:

Open in Colab

Choose Sample Text

The patient is a 40-year-old white male who presents with a chief complaint of "chest pain". The pat...

Text annotated with identified Named Entities

The patient is a 40-year-old white male who presents with a chief complaint of "chest pain". The patient is diabetic and has a prior history of coronary artery disease. The patient presents today stating that his chest pain started yesterday evening and has been somewhat intermittent. He has been advised Aspirin 81 milligrams QDay. Humulin N insulin 50 units in a.m.

HCTZ 50 mg QDay. Nitroglycerin 1/150 sublingually PRN chest pain.

Finance NLP

- Extracting financial entities from annual reports, as Expenses, Loses, Profit declines or increases, etc.

Choose the pretrained model to test

finner_financial_small

Entity labels

PERCENTAGE x

PROFIT_DECL... x

FISCAL_YEAR x

CURRENCY x

AMOUNT x

Choose Sample Text

License fees revenue decreased 40 %, or \$ 0.5 million to \$ 0.7 million for the year ended December 31, 2020 compared to \$ 1.2 million for ...

Text annotated with identified Named Entities

License fees revenue decreased 40 %, or \$ 0.5 million to \$ 0.7 million for the year ended December 31, 2020 compared to \$ 1.2 million for the year ended December 31, 2019

PROFIT_DECLINE PERCENTAGE CURRENCY AMOUNT CURRENCY AMOUNT FISCAL_YEAR CURRENCY AMOUNT FISCAL_YEAR

Healthcare NLP

- Exploring oncological terms.

Choose the Task :

Named Entity Recognition

Choose the Pretrained Model :

ner_oncology_wip

Entity labels

CANCER_SUR... x

GRADE x IMAGING_TEST x

TUMOR_FINDI... x

DIRECTION x GENDER x

SITE_BREAST x

CANCER_DX x

Choose Sample Text

The patient was operated. The breast conserving surgery was performed. The clip in the right breast was localized by the ROLL method ...

Text annotated with identified Named Entities

The patient was operated. The **breast conserving surgery** was performed. The clip in the **right** **breast** was localized by the **ROLL** method under **X-ray** guidance. The **lesion** in **her** **left** **breast** was localized by the **ROLL** method under **US guidance**. The pathologic results were the remnant foci of **high-grade** **DCIS in the right breast**.

CANCER_SURGERY **DIRECTION** **SITE_BREAST** **CANCER_SURGERY** **IMAGING_TEST** **TUMOR_FINDING** **GENDER** **DIRECTION** **SITE_BREAST** **CANCER_SURGERY** **IMAGING_TEST** **GRADE** **CANCER_DX**

Fashion NLP

- recognize fashion concepts such as *brands*, *colors* or *seasons* in text, or to classify whole text documents into one or more categories.

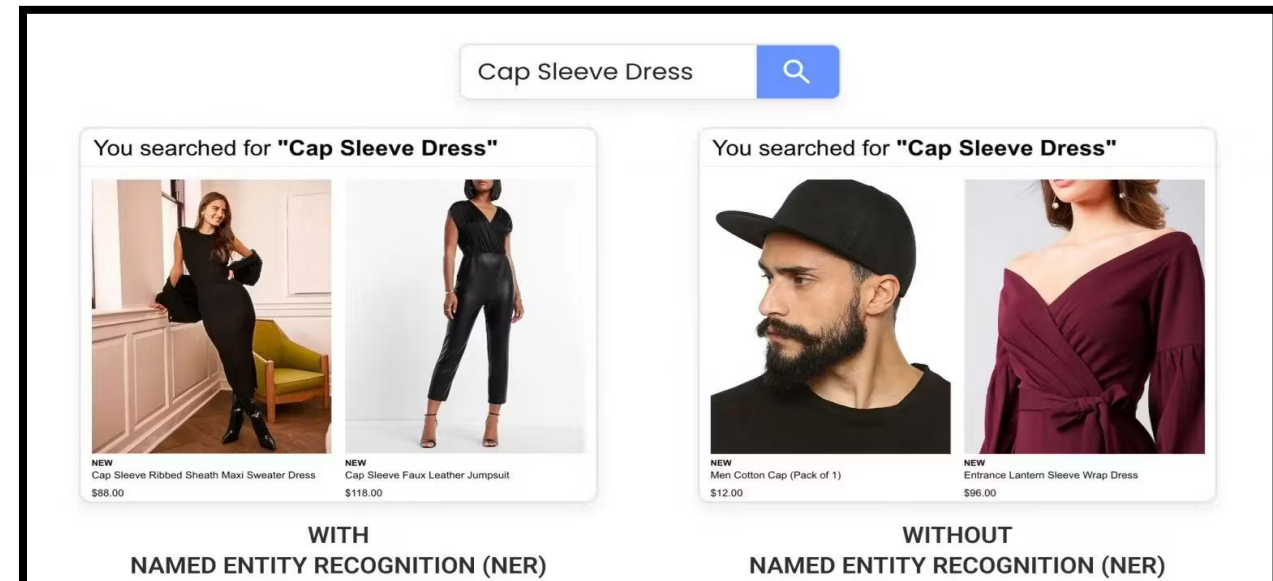
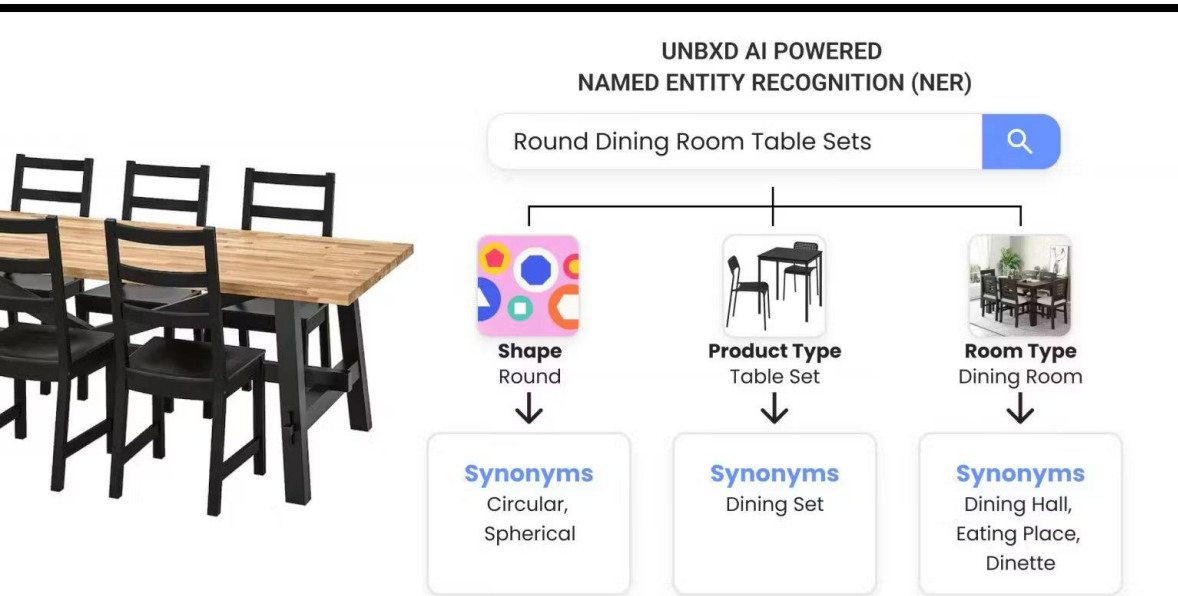
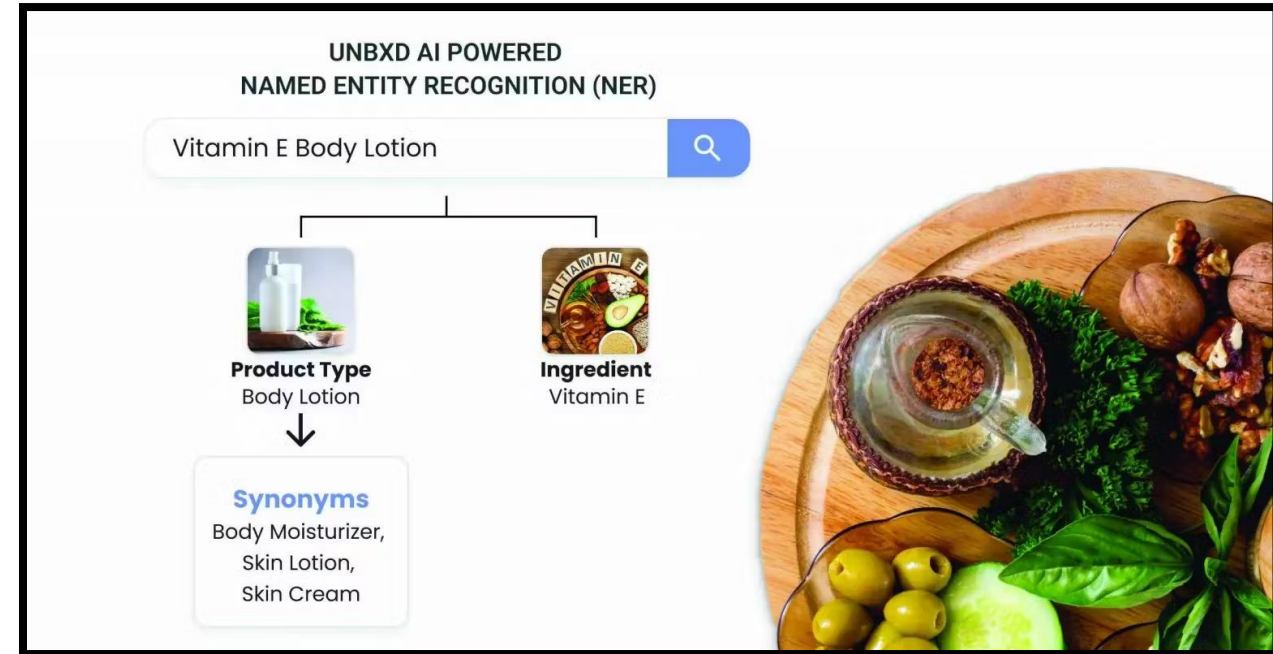
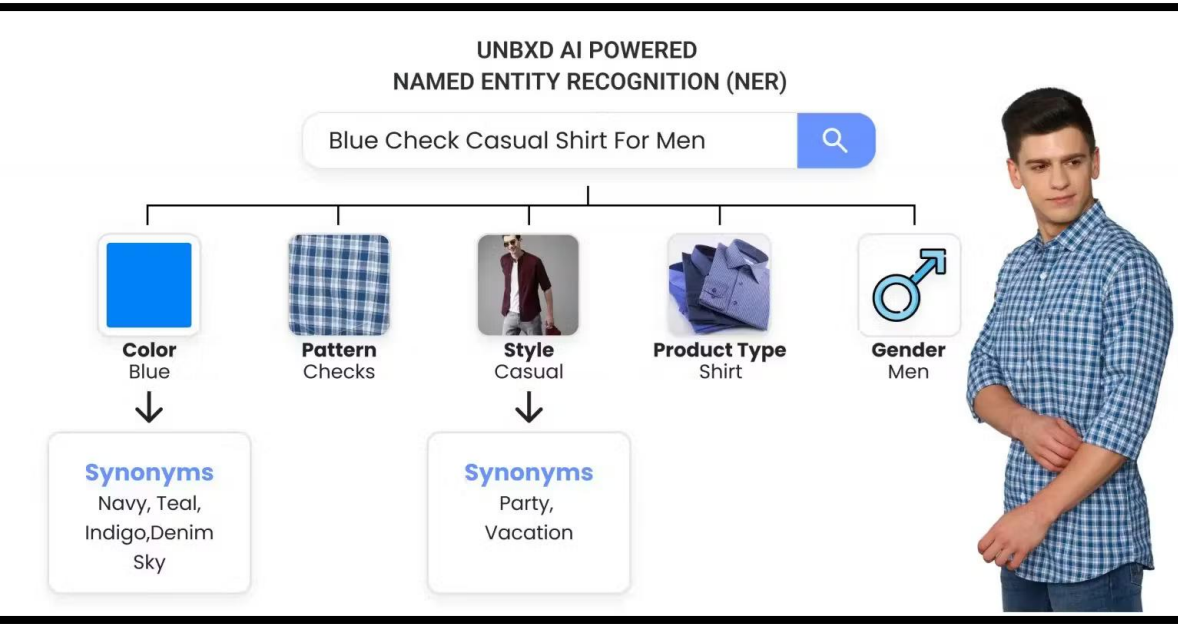
Back in 2000 , **People Magazine** PUBLISHER highlighted **Prince Williams'** PERSON style who at the time was a little more fashion-conscious , even making fashion statements at times .

Now-a-days the prince mainly wears **navy** COLOR **suits** ITEM (sometimes **double-breasted** DESIGN) , **light blue** COLOR **button-ups** ITEM with **classic** LOOK **pointed** DESIGN **collars** PART , and **burgundy** COLOR **ties** ITEM .

But who knows what the future holds ...

Duchess Kate PERSON did wear an **Alexander McQueen** BRAND **dress** ITEM to the **wedding** OCCASION in the **fall of 2017** SEASON .

Search Engine & recommendation NER



Steps in NER

Extracting the Information

- The first step in NER is extracting the information by detecting and preparing the named entities mentioned in the document, paragraph, sentence and texts. The entire extraction phase includes the tagging the speech, detecting the boundaries of sentence, capitalization rules and co-reference in the documents that more important to use and find more specific terms in the search.

Searching the Entities

- The next process in NER is searching the entity candidates to mention in the document. Names, multiple pages, and informative web pages pseudonyms are also considered to catch the synonyms. The searcher maintains the balance with accuracy and recall the right entity while keeping a small set of entities in order to reduce the calculation needed for recognizing such entities.

Tesla ORG last Monday DATE

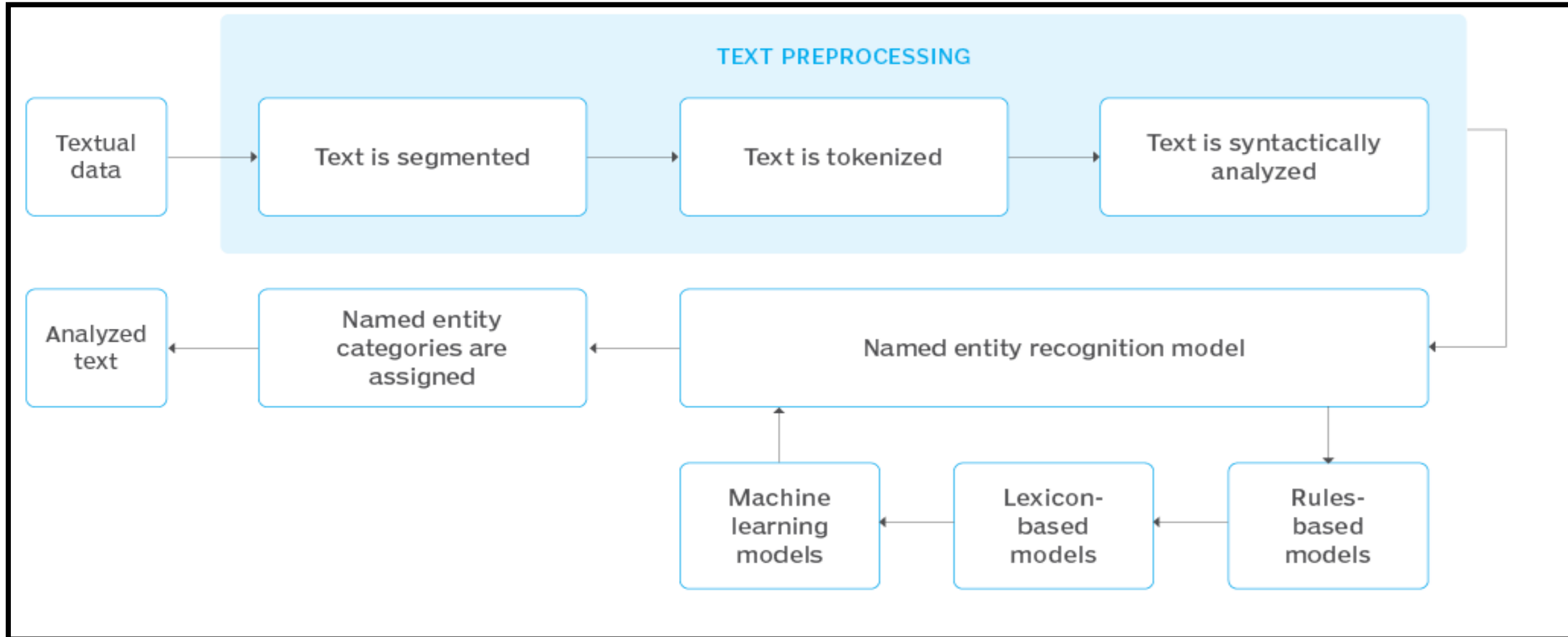
announced it will revamp its top-

selling Model Y electric car. PRODUCT

For most of their history on Wall Street, LOC computers have been strictly quantitative-dividing, say, prices by earnings and ranking the results. But that is destined to change. A dramatic demonstration of silicon's verbal potential came in 2011, DATE when an IBM ORG system called Watson ORG bested two human champions at Jeopardy! To accomplish this feat the computer had to grasp not just numbers but genealogical relationships, time, proximity, causality, taxonomy and a lot of other connections.

Last year EquiBot ORG's software picked up a buzz around Amarin Corp ORG., an Irish LOC drug company with a prescription-only diet supplement that uses omega-3 fatty acids. The international ETF got in below \$3, Price well before the regulatory nod that sent the stock to \$15 Price. Another move involved adding Visa ORG to the domestic fund after the system measured ripples leading from announcements of chain-store closings toward higher credit card volume.

NER based preprocessing workflow



Fine-Grained NER

Basic NER Entity Category

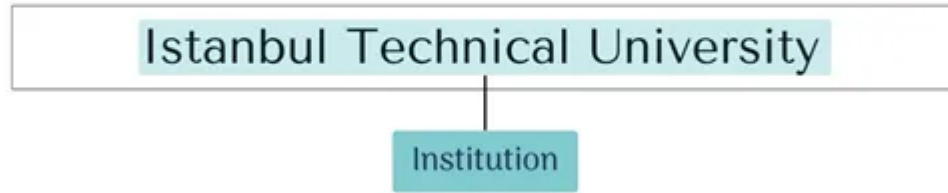
Named Entity Type
ORGANIZATION
PERSON
LOCATION
DATE
TIME
MONEY
PERCENT
FACILITY
GPE

person actor architect artist athlete author coach director	doctor engineer monarch musician politician religious_leader soldier terrorist	organization airline company educational_institution fraternity_sorority sports_league sports_team	terrorist_organization government_agency government political_party educational_department military news_agency
location city country county province railway road bridge	body_of_water island mountain glacier astral_body cemetery park	product engine airplane car ship spacecraft train	camera mobile_phone computer software game instrument weapon
			art film play event attack election protest
written_work newspaper music	military_conflict natural_disaster sports_event terrorist_attack	website broadcast_network broadcast_program tv_channel currency stock_exchange algorithm programming_language transit_system transit_line	
building airport dam hospital hotel library power_station restaurant sports_facility theater	time color award educational_degree title law ethnicity language religion god	chemical_thing biological_thing medical_treatment disease symptom drug body_part living_thing animal food	

Basic Problems in NER

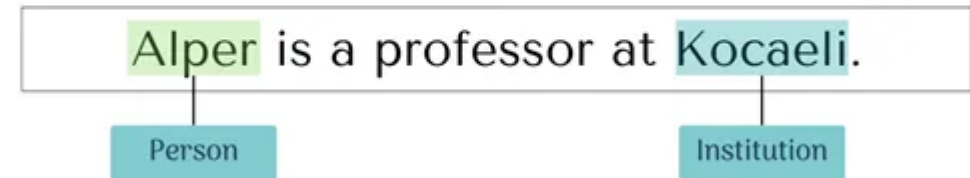
- Generative in nature
- Variation of NEs – e.g. Prof Manning, Chris Manning, Dr Chris Manning
- Ambiguity of NE types:
 - **Washington** (location vs. person)
 - **May** (person vs. month)
 - **Ford** (person vs organization)
 - **1945** (date vs. time)
- Ambiguity with common words, e.g. “*Kabita*”
 - Name of person vs. poem

- Detecting a certain entity in a text which may contain another entity



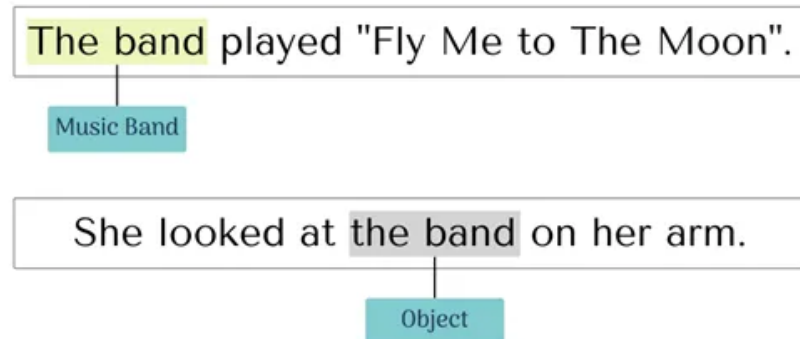
"Istanbul Technical University" specifies a single entity, disregarding that "Istanbul" is another entity. NER model can understand the discourses in texts thanks to NLP and can handle nested entities, as well.

- Detecting correct types of entities that have a common lexicon with multiple entity types



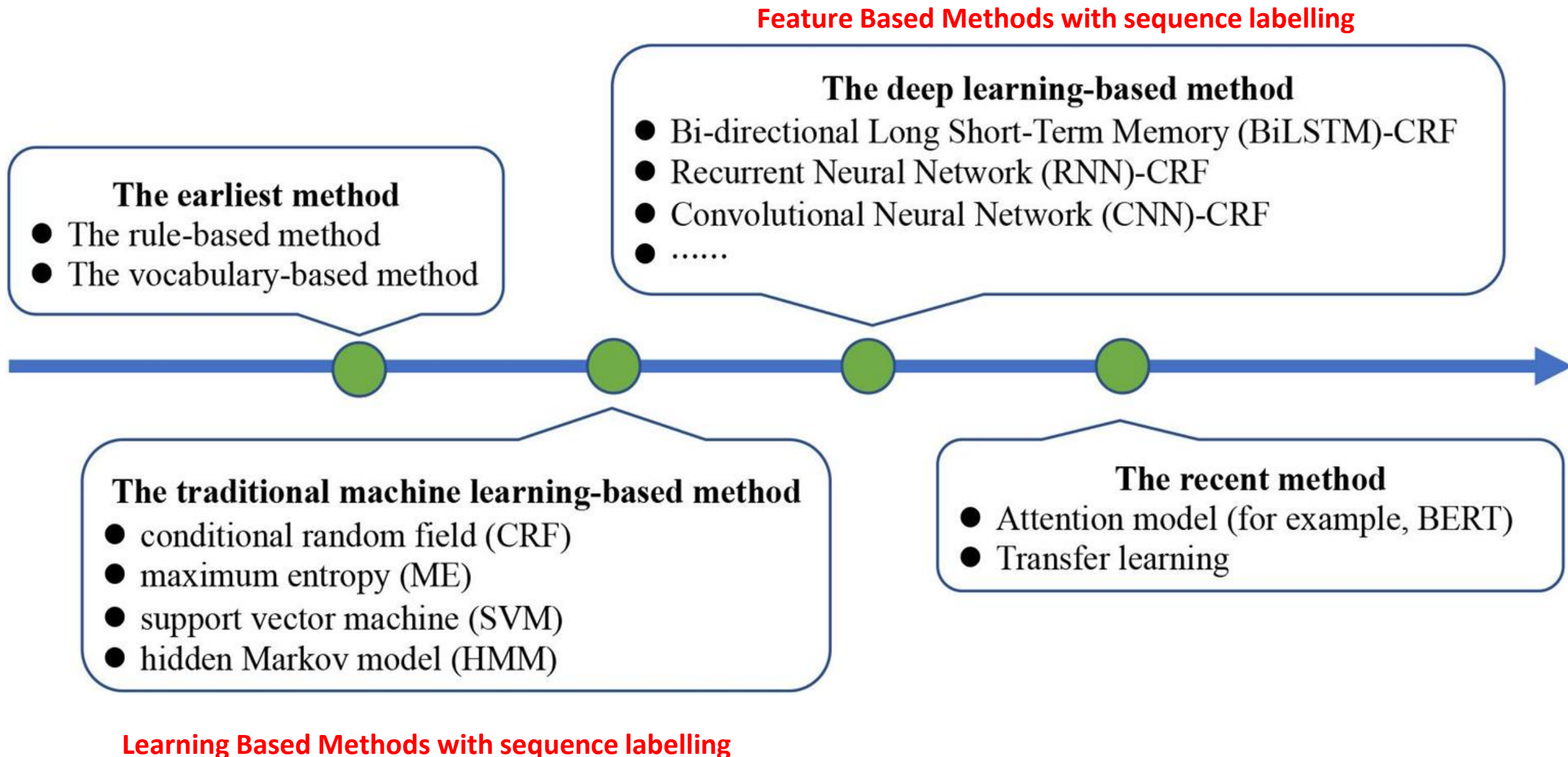
The word which is marked as Institution is the University of Kocaeli, not Kocaeli as a city. NER models can extract types of entities from sentence structure.

- Detecting correct types of entities which include homonym phrases



"The band" in these sentences is not from the same entity type. NER models use Relation Extraction to detect entities from the overall meaning of the text.

Evolution and types of NER Methodology



Different NER systems

Dictionary-based systems

- This is the simplest NER approach. Here we will be having a dictionary that contains a **collection of vocabulary**. In this approach, **basic string matching algorithms** are used to check whether the entity is occurring in the given text to the items in the vocabulary. The method has limitations as it is required to update and maintain the dictionary used for the system.

Dictionary Based NER

PERSON	
Names of famous people	

Pierre Vinken , 61 years old , will join the board as a nonexecutive director Nov. 29 .

DATE	
Month names and abbrs.	

Rule-based systems

- Here, the model uses a pre-defined set of rules for information extraction. Mainly two types of rules are used, **Pattern-based rules**, which depend upon the morphological pattern of the words used, and **context-based rules**, which depend upon the context of the word used in the given text document.

- Eg: Create regular expressions to extract:
 - Telephone number
 - E-mail
 - Capitalized names

Context based rules

- [PERSON] **earns** [MONEY]
- [PERSON] **joined** [ORGANIZATION]
- [PERSON] **left** [ORGANIZATION]
- [PERSON] **joined** [ORGANIZATION] as [JOBTITLE]
- [ORGANIZATION]'s [JOBTITLE] [PERSON]
- [ORGANIZATION] [JOBTITLE] [PERSON]
- **the** [ORGANIZATION] [JOBTITLE]
- **part of the** [ORGANIZATION]
- [ORGANIZATION] headquarters in [LOCATION]
- **price of** [ORGANIZATION]
- **sale of** [ORGANIZATION]
- **investors in** [ORGANIZATION]
- [ORGANIZATION] **is worth** [MONEY]
- [JOBTITLE] [PERSON]
- [PERSON], [JOBTITLE]



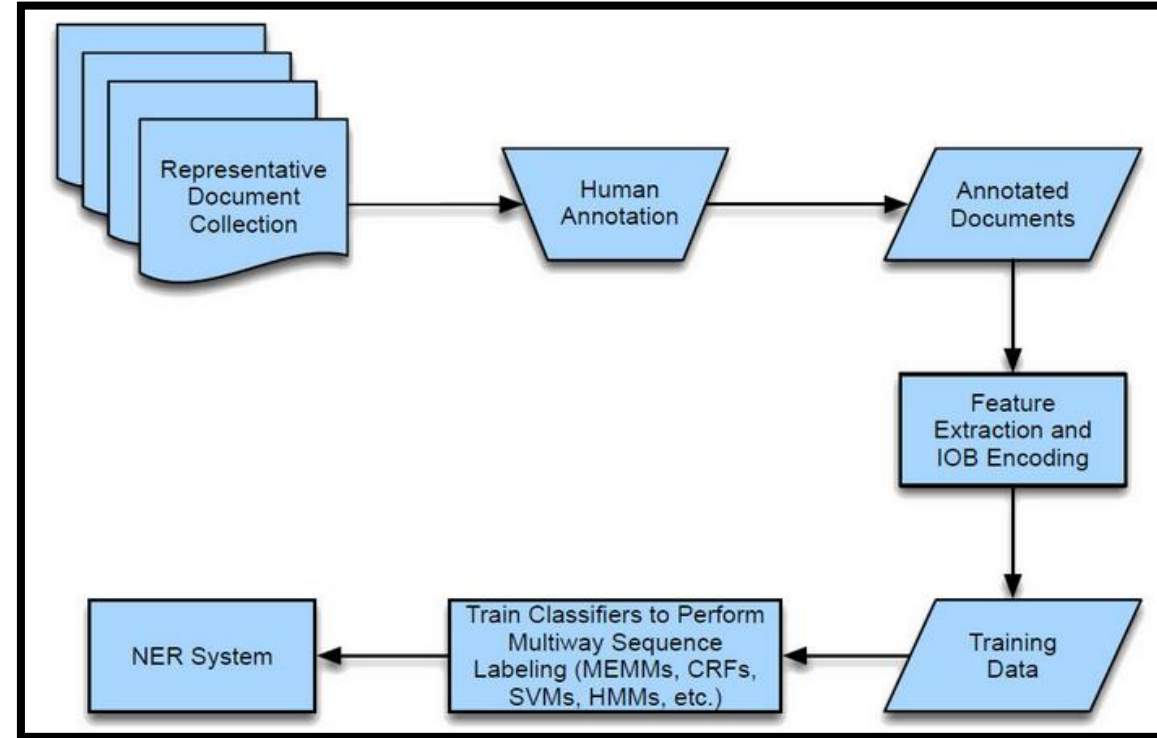
Pattern Based

When context-based rules are useful?

- Use of context-based patterns is helpful in ambiguous cases
- “Ratan Tata ” and “Tata Sons” are indistinguishable
- But with the phrase “Ratan Tata of Tata Sons” and the Person entity “Ratan Tata” recognised, we can use the pattern “[*Person*] of [*Organization*]" to identify “Tata Sons” correctly

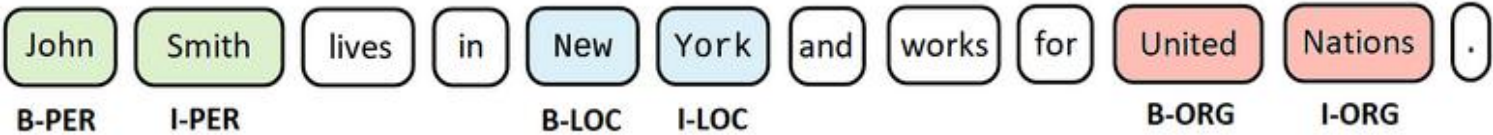
Sequence Labelling

- Sequence labeling is a task that assigns categorical label to each element in an input sequence. Many problems can be formalized as sequence labeling task, including speech recognition, video analysis and various problems in NLP (e.g. POS tagging, NER, Chunking, etc.).
- Traditionally sequence labeling requires large amount of hand-engineered features and domain-specific knowledge, but recently neural approaches have achieved state-of-the-art performance on several sequence labeling benchmarks. A common data format for sequence labeling task is IOB (Inside-Outside-Beginning),



The IOB format (short for inside, outside, beginning) is a tagging format that is used for tagging tokens in a chunking task such as named-entity recognition. These tags are similar to part-of-speech tags but give us information about the location of the word in the chunk.

- BNP:** the beginning of a noun phrase
- I-NP:** describes that the word is inside of the current noun phrase.
- O:** end of the sentence.
- B-VP and I-VP:** beginning and inside of a verb phrase.



Statistical models:

- Maximum Entropy Markov Models (MEMMs)
- Conditional Random Fields (CRFs)

Neural models:

- Recurrent networks (or transformers) that predict a label at each time step, possibly with a CRF output layer.

Word	Label	Tag
American	B	ORG
Airlines	I	ORG
a	O	–
unit	O	–
of	O	–
AMR	B	ORG
Corp.	I	ORG
immediately	O	–
matched	O	–
the	O	–
move	O	–
spokesman	O	–
Tim	B	PERS
Wagner	I	PERS
said	O	–

Original Input Sentence

Pierre Vinken , 61 years old , will join IBM 's board
as a nonexecutive director Nov. 29 .



POS Tags

Pierre_NNP Vinken_NNP ,_, 61_CD years_NNS old_JJ ,_,
will_MD join_VB IBM_NNP 's_POS board_NN as_IN a_DT
nonexecutive_JJ director_NN Nov._NNP 29_CD ._.



Chunks

[NP Pierre Vinken] , [NP 61 years] old , will join
[NP IBM] 's [NP board] as [NP a nonexecutive director]
[NP Nov. 2] .



IOB NER

Pierre_B-PERS Vinken_I-PERS ,_O 61_O years_O old_O ,_O
will_O join_O IBM_B-ORG 's_O board_O as_O a_O
nonexecutive_O director_O Nov._B-DATE 29_I-DATE ._O

Workflow of Sequence Modeling

Training

1. Collect a set of representative training documents
2. Label each token for its entity class or other (O)
3. Design feature extractors appropriate to the text and classes
4. Train a sequence classifier to predict the labels from the data

Testing

1. Receive a set of testing documents
2. Run sequence model inference to label each token
3. Appropriately output the recognized entities

Part of speech:

NP NP RB VBD IN NP NP , CC PRP VBZ RB VBG PRP IN PRP .
Mrs. Clinton previously worked for Mr. Obama, but she is now distancing herself from him .

Named entity recognition:

Person Date Person Date
Mrs. Clinton previously worked for Mr. Obama, but she is now distancing herself from him.

Co-reference:

Coref Coref Coref
Mention Ment M Mention M
Mrs. Clinton previously worked for Mr. Obama, but she is now distancing herself from him.

Basic dependencies:

compound nsubj cc conj nmod case compound nsubj aux advmod nmod dobj case
NP NP RB VBD IN NP NP , CC PRP VBZ RB VBG PRP IN PRP .
Mrs. Clinton previously worked for Mr. Obama, but she is now distancing herself from him .

Complete NLP Preprocessing Pipeline — Python

Unit 1 Recap

1. Tokenize

▶ "I love NLP!" →
['I', 'love', 'NLP', '!']

2. Lowercase

▶ → ['i', 'love', 'nlp', '!']

3. Remove Punct.

▶ → ['i', 'love', 'nlp']

4. Remove Stops

▶ → ['love', 'nlp']

5. Lemmatize

▶ → ['love', 'nlp']

6. POS Tag

▶ → [('love', 'VB'), ('nlp', 'NN')]

7. NER

▶ → [] (no entities found)

python — end-to-end NLP pipeline

```
import nltk, spacy, re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

def nlp_pipeline(text):
    # Step 1-4: clean & tokenize
    text = text.lower()
    text = re.sub(r'^\w\s', '', text)
    stop = set(stopwords.words('english'))
    tokens = [w for w in text.split()
               if w not in stop]

    # Step 5: lemmatize
    lem = WordNetLemmatizer()
    tokens = [lem.lemmatize(w) for w in tokens]

    # Step 6-7: POS + NER via spaCy
    nlp = spacy.load('en_core_web_sm')
    doc = nlp(' '.join(tokens))
    return doc
```

Unit 1 — Key Takeaways

01

Language & NLP

NLP bridges human language and AI; challenges include ambiguity, context, and sarcasm.

02

NLP Applications

Opinion mining, dialog, MT, search, summarization — NLP powers them all.

03

5 Phases of NLP

Lexical → Syntactic → Semantic → Discourse → Pragmatic.

04

Preprocessing

Lowercase, remove punctuation/stops, normalize — foundations of every pipeline.

05

Tokenization

Word / sentence / subword / character. BPE used by Transformers.

06

Stemming & POS

Porter/Snowball stem; WordNet lemmatize. POS: NN, VBZ, JJ, DT, IN.

07

NER

spaCy/NLTK identify PERSON, ORG, DATE, LOCATION — basis of IE systems.

Questions & Discussion



Next class: Unit 2 — Feature Extraction (BoW, TF-IDF, N-grams, Naïve Bayes)